

GamutX

Branching Workflow

GIT

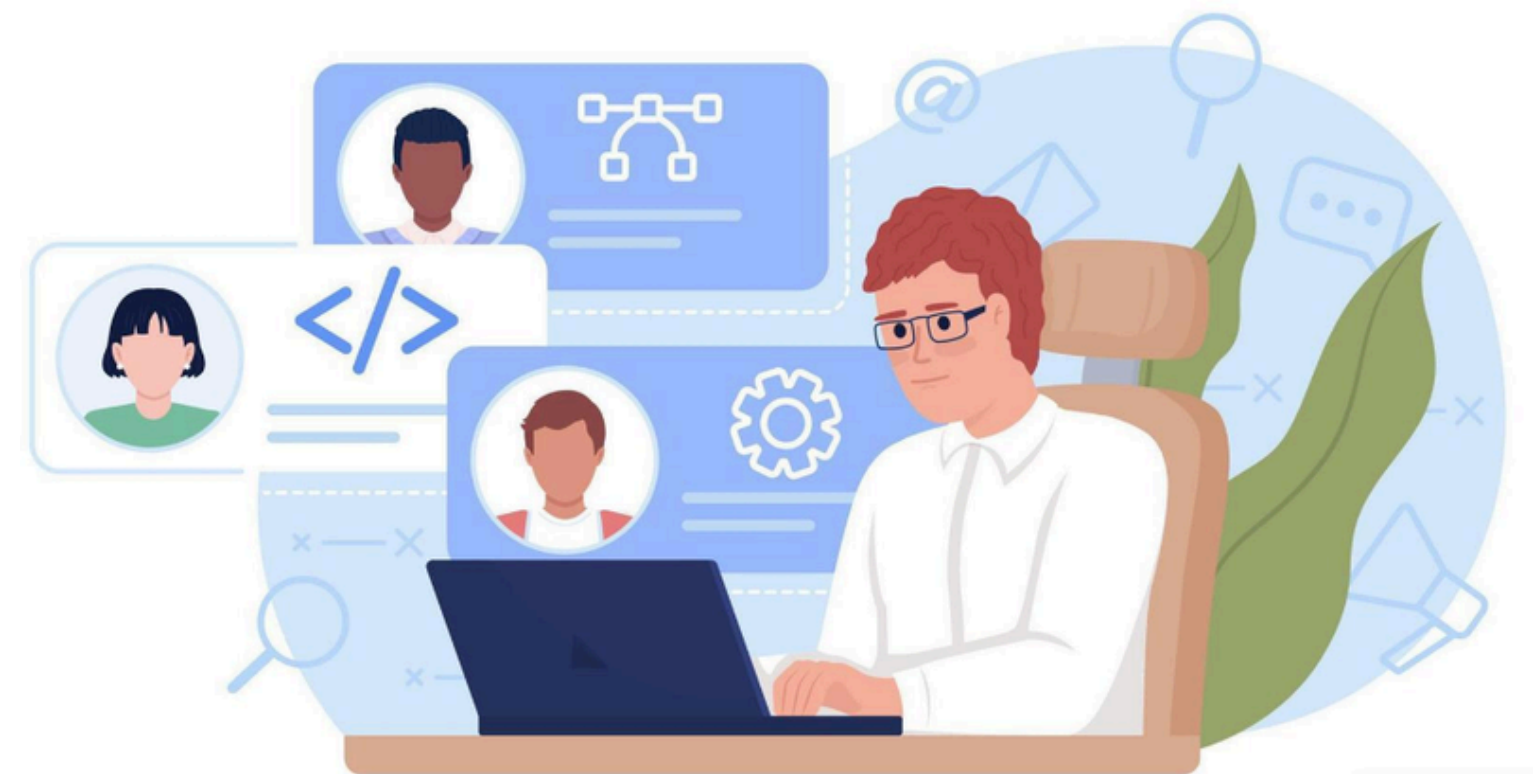
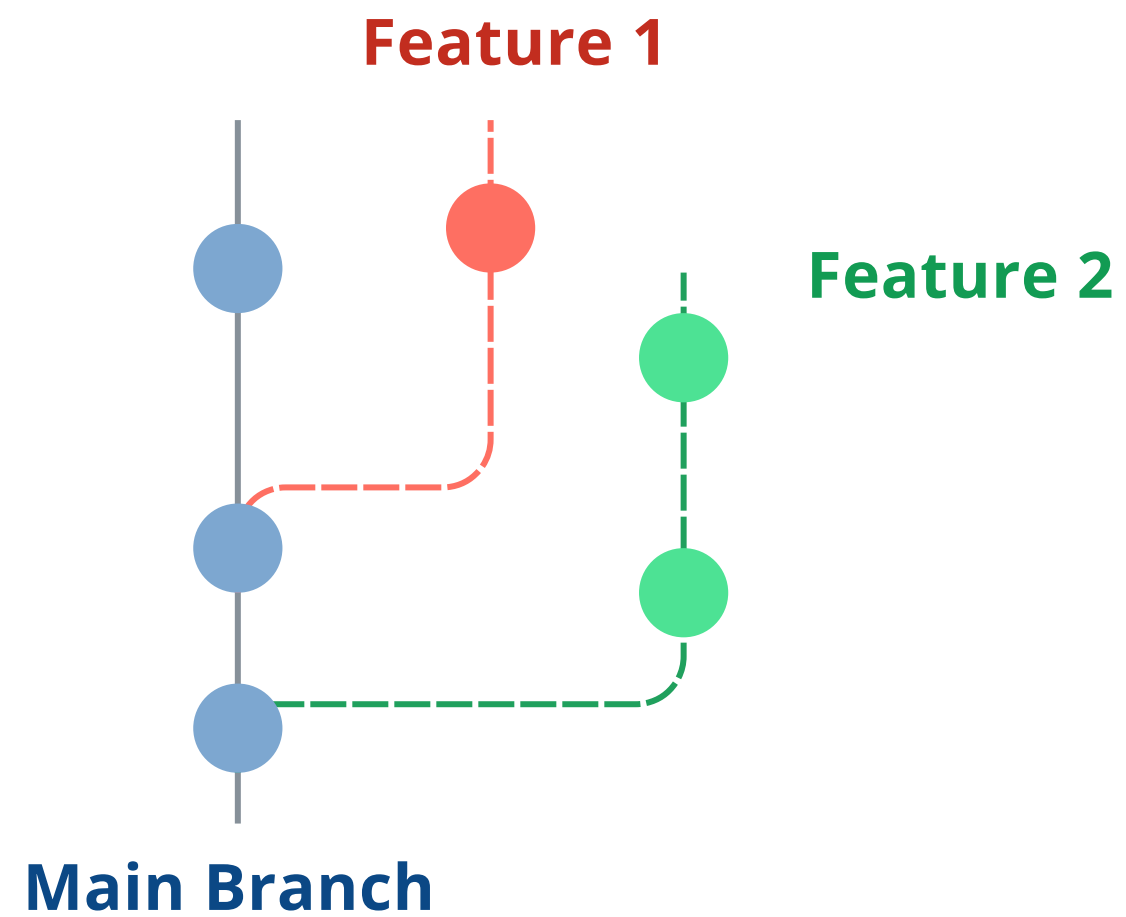


GIT Branching Workflow

- What is a Collaborative Workflow?
- Why is Collaborative Workflow Important?
- What Are Git Branches?
- Why Are Branches Important?
- What is the Default Branch?
- Checking Which Branch You Are On in a Local Repo
- Creating a New Branch in VS Code
- Git Workflow on a New Branch
- Understanding Staged vs Unstaged Files
- Viewing Branches on Remote (GitHub)
- Industry Best Practices for Branches

What is a Collaborative Workflow?

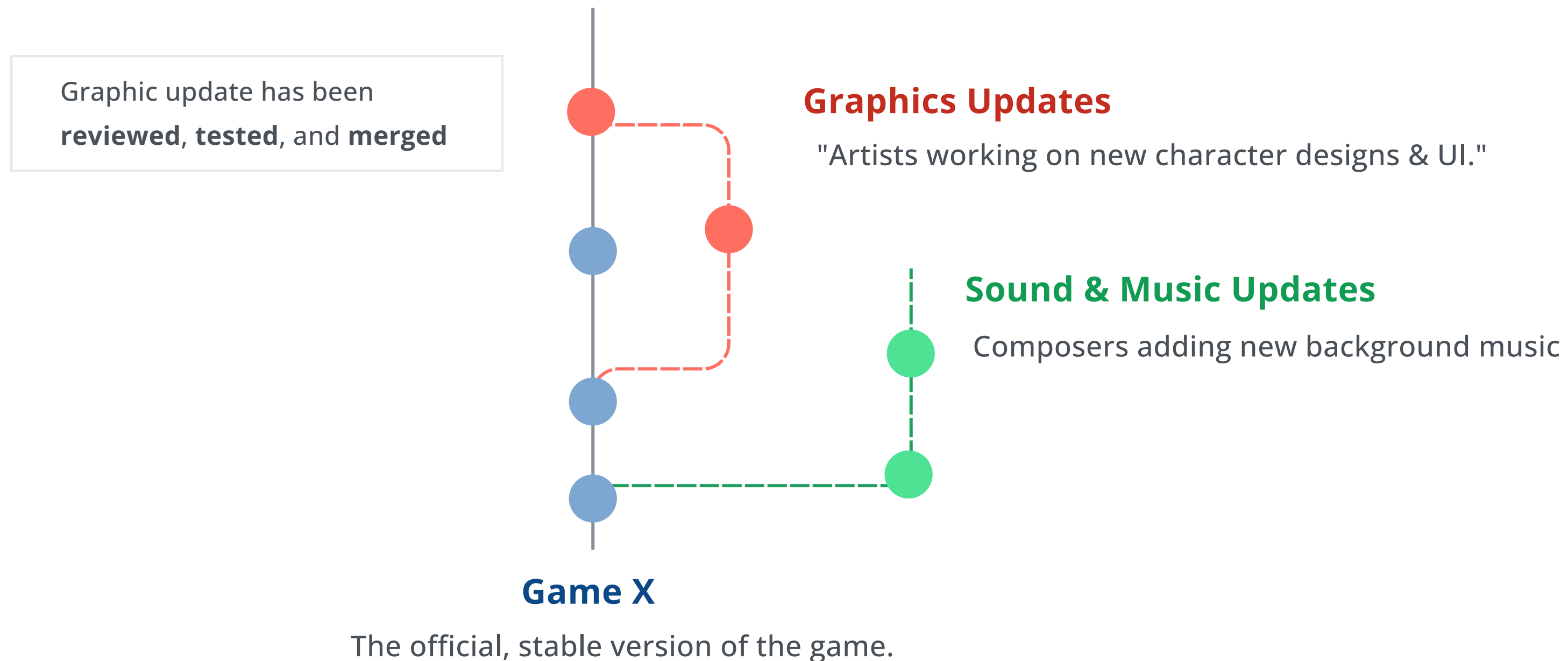
- A process where **multiple developers work on different branches of the same project.**
- Changes are merged through **Pull Requests (PRs)** to ensure **review and approval.**
- Prevents **conflicts**, **maintains code quality**, and **avoids breaking the main codebase.**



What Are Git Branches?

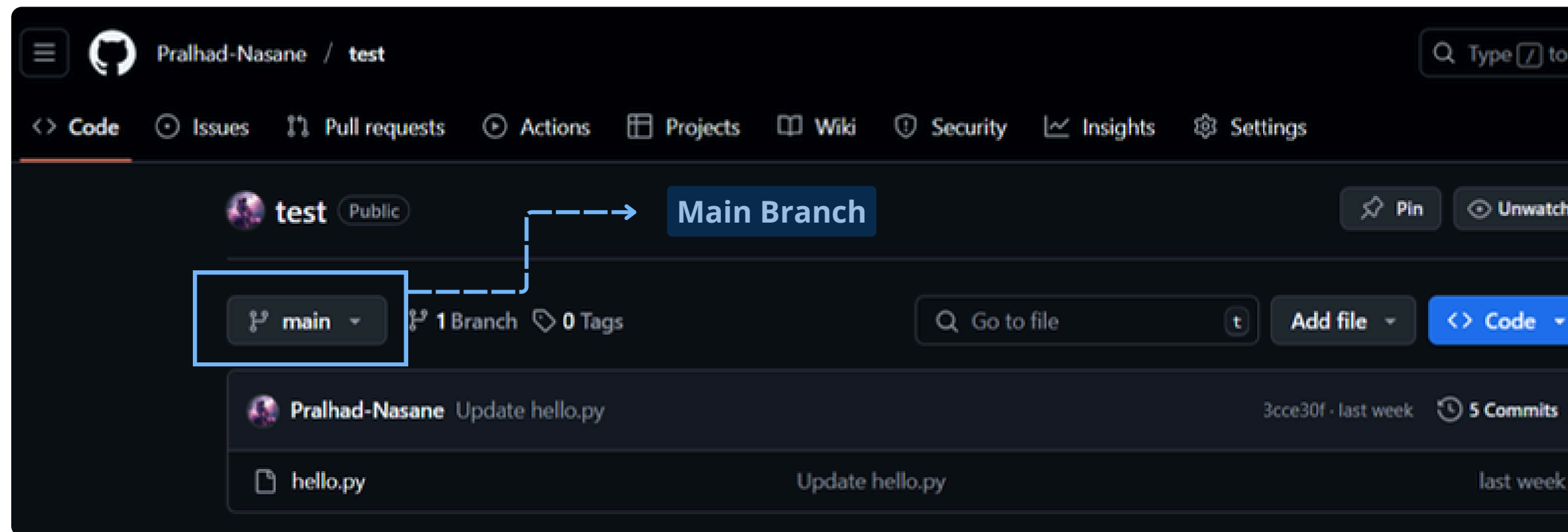
- A Git branch is **like a separate workspace inside your project.**
- It lets you work on **new features, bug fixes,** or **experiments without affecting the main project.**

Imagine you and your team are developing a video game:



What is the Default Branch?

- Every Git repository **starts with a default branch**, usually named **main** (or sometimes **master**).
- This is the **primary branch where the final, stable version of the code lives**.



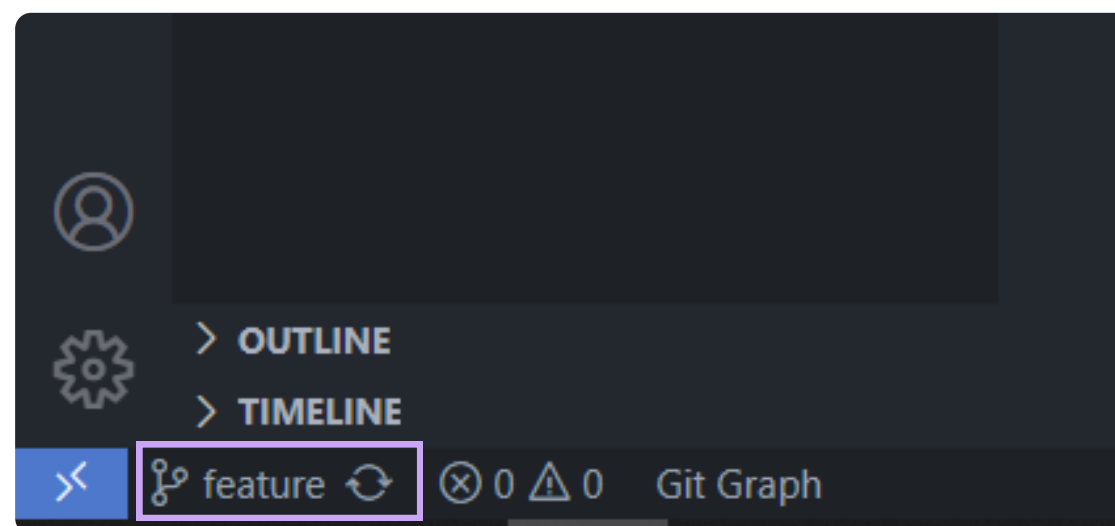
Best Practice

- ✗ Never **commit directly** to the **main branch** in production or team projects!
- ✓ Always **use feature branches** and **merge changes via Pull Requests (PRs)** after review.

Checking Which Branch You Are On in a Local Repo

Check Your Active Branch

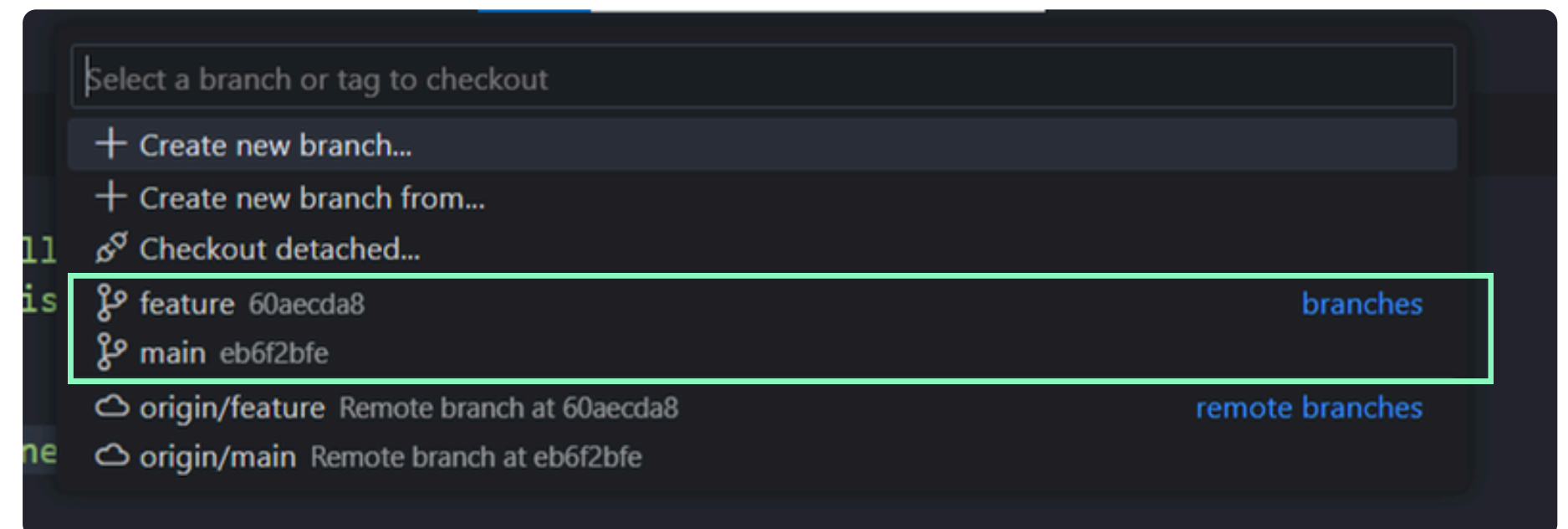
- The current branch is displayed
- in the status bar at the **bottom-left of the VS Code window.**



Branch

Use the Source Control Tab

- Click on the **branch name** in the **bottom-left status bar**.
- A **list of branches** will appear.
- Select the **branch you want to switch to**.



🚀 Now you can easily check and switch branches in VS Code.

Creating a New Branch in VS Code

Let's See How We can **Create a Branch** using **V S Code**

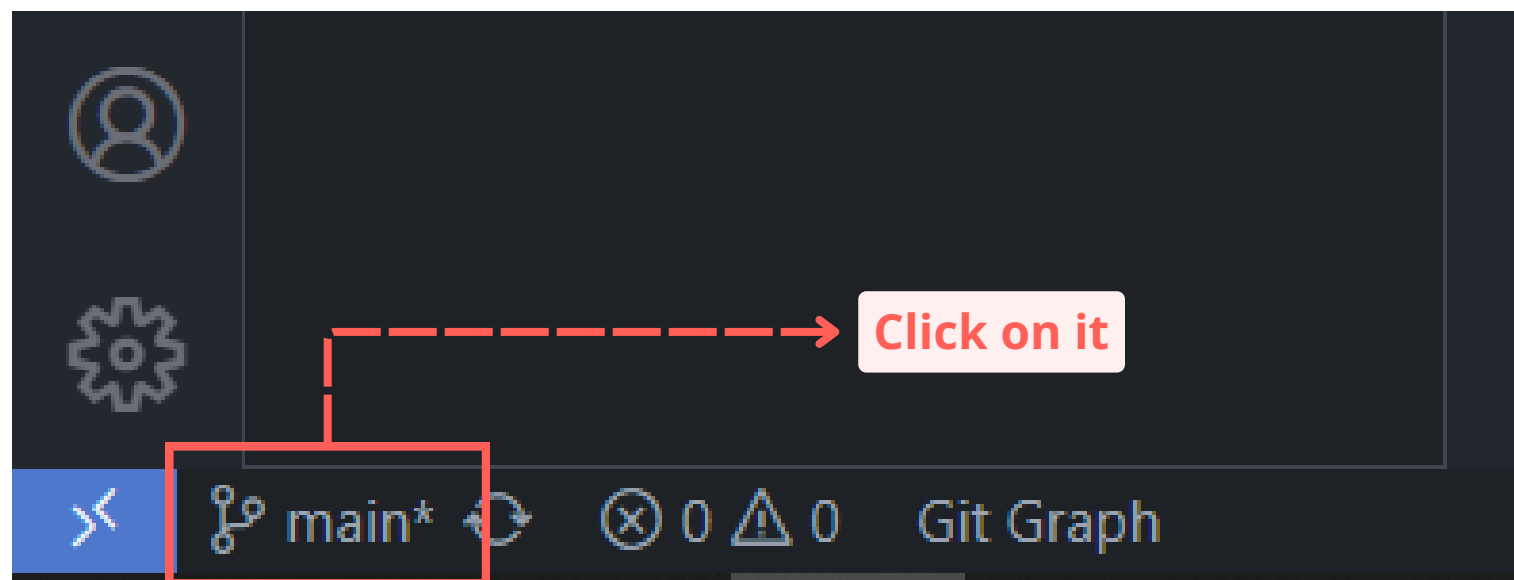
1. Check Your Active Branch

Open Your Repository

- Launch **VS Code** and open your Git-tracked project.

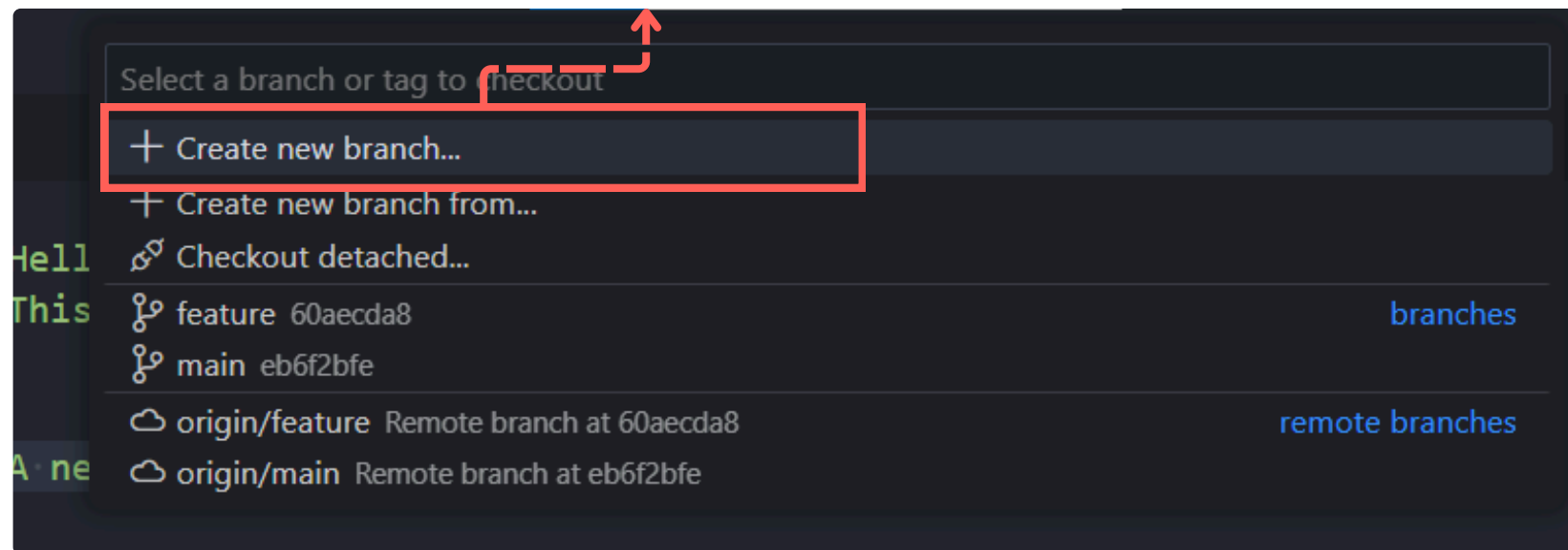
Click on the Active Branch

- Look at the **bottom-left corner of the VS Code** window.
- **Click** on the **current branch** name (e.g., main).



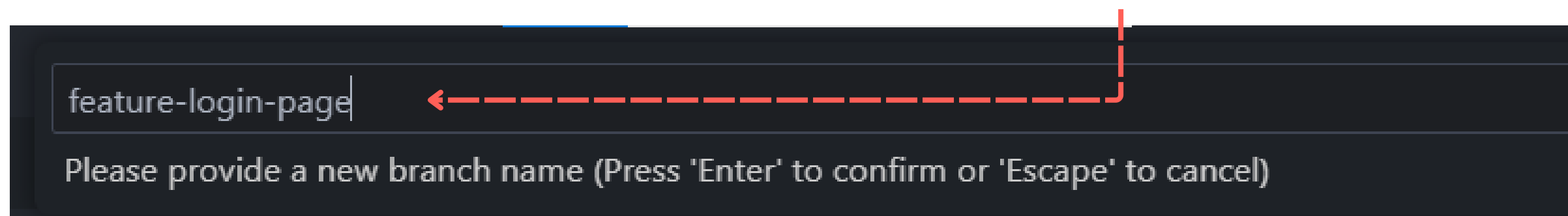
2. Select "Create Branch"

- A **dropdown menu** will appear.
- Click on "**Create New Branch**".



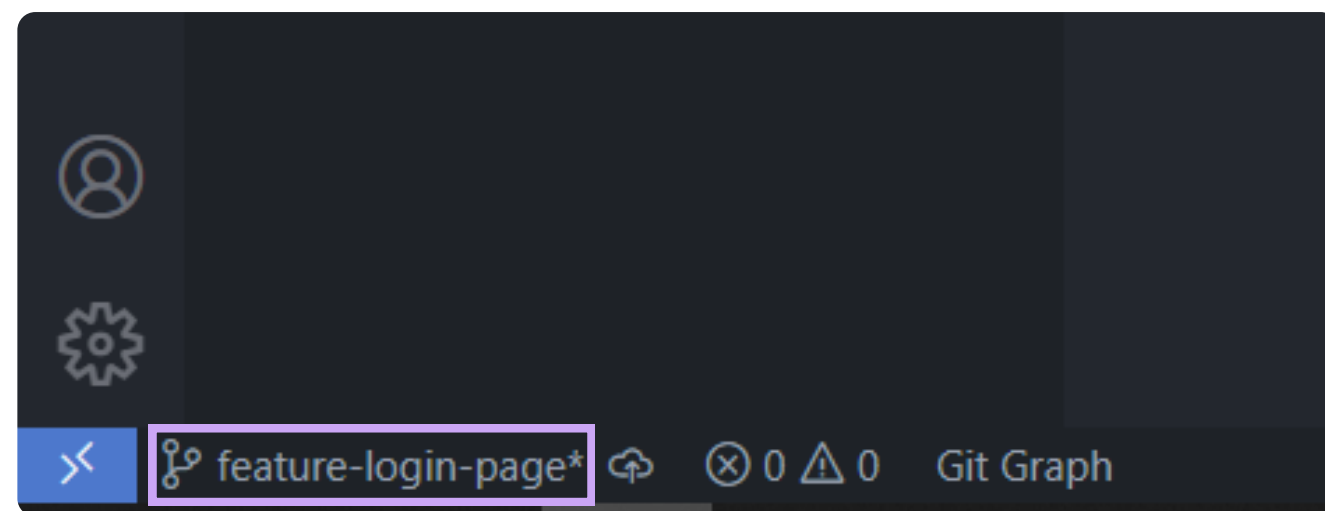
3. Enter a Branch Name

- Type a **descriptive name** for your branch e.g., **feature-login-page**

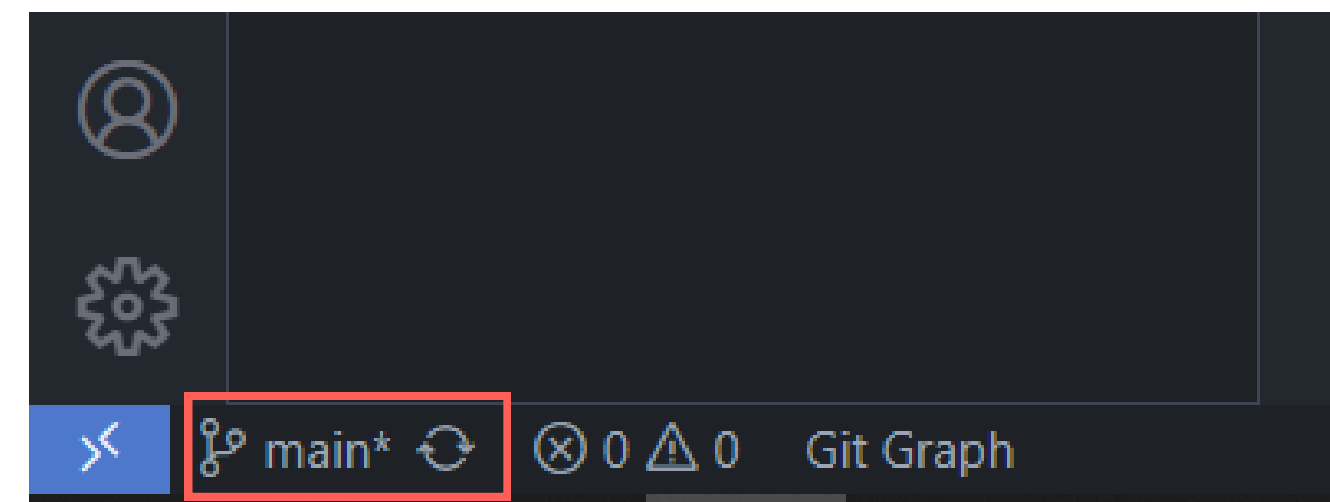


4. Switch to the New Branch

- VS Code will **automatically switch to the new branch**.
- The branch name in the **status bar** will update.



After Switch to new branch



Before Switching

What Happens When You Create a New Branch?

- ✓ The **new branch is a copy of the main branch** at that moment
- ✓ You can now make changes **without affecting the main branch**
- ✓ This lets you **develop features or fix bugs separately** while **keeping the main project stable!**

Git Workflow on a New Branch

GamutX

File Saving

Create a File

Open the File

Work on Edits

Save the File

Share the File

GIT WorkFlow

Create a New Branch

Switch to the New Branch

Make Changes in Code

Stage & Commit Changes

Push to Remote



Git Workflow on a New Branch

1. Add Files

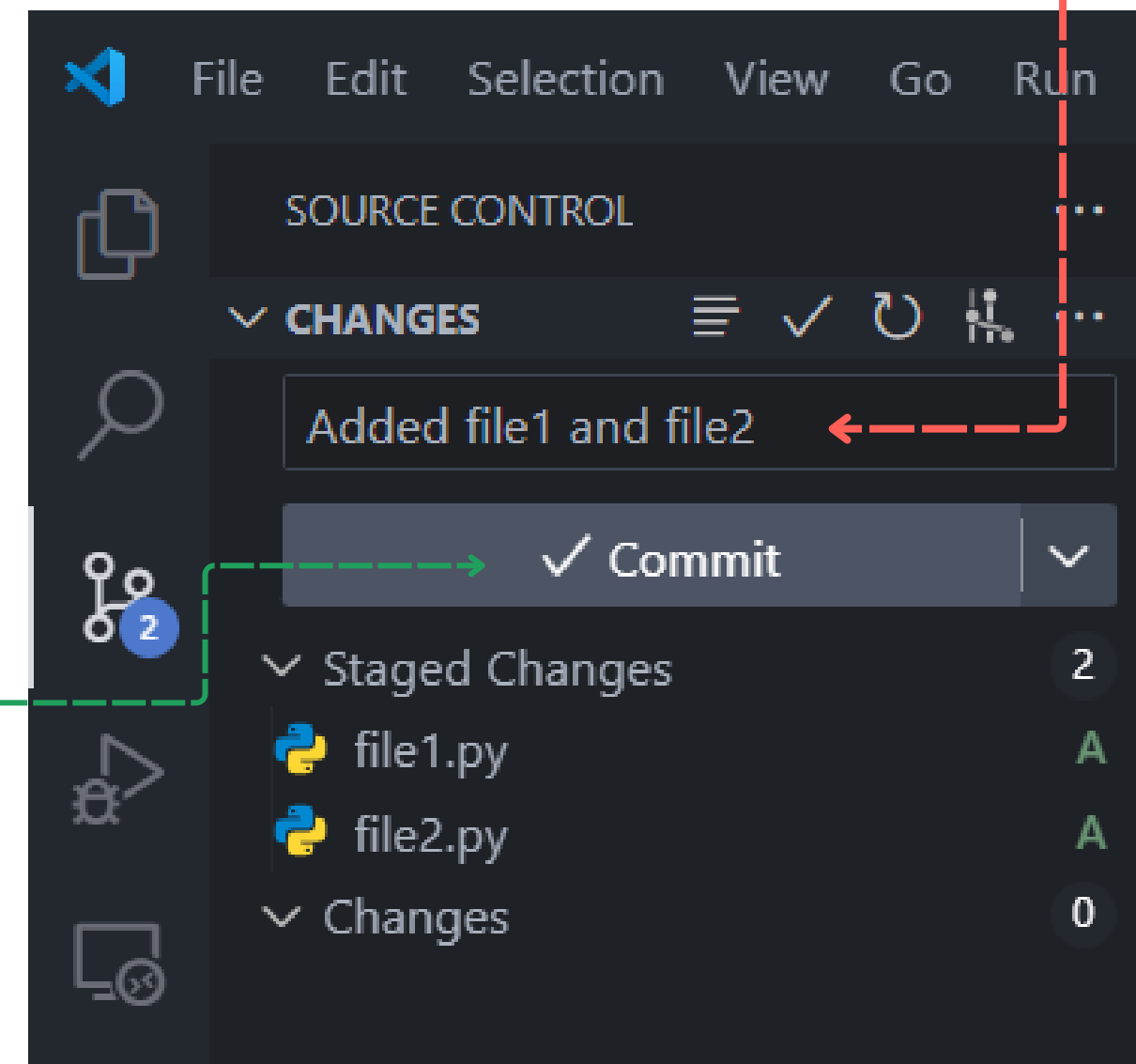
- In your **Feature branch**,
 - create new files . **file1.py, file2.py**

```
# file1.py
print("Hello from file1!")

# file2.py
print("Hello from file2!")
```

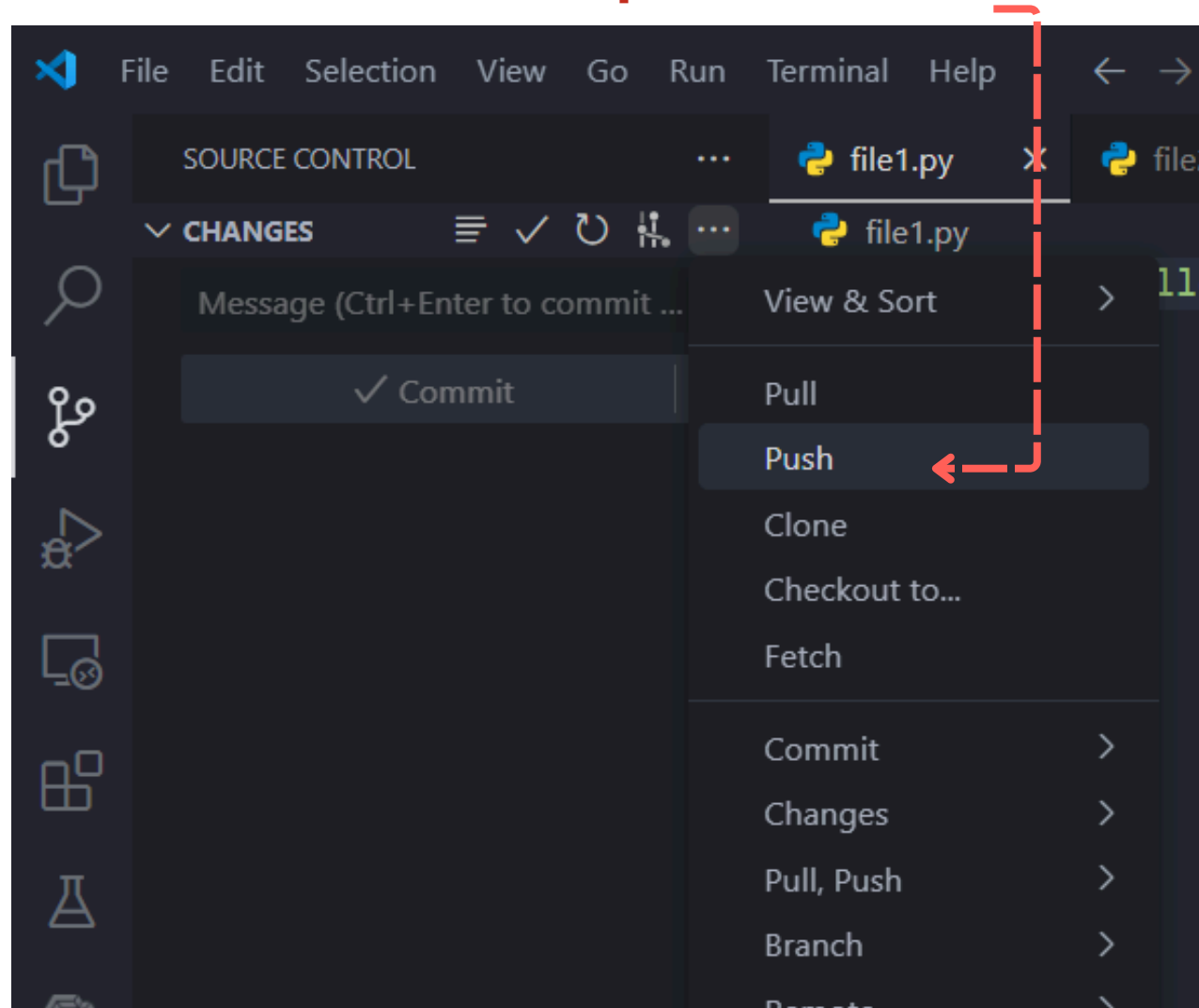
2. Stage & Commit Changes

- Open Source Control (Ctrl + Shift + G).
- **Click the + button next to each file** to stage them.
- Write a commit message **"Added file1 and file2"**



3. Push the Changes to Remote

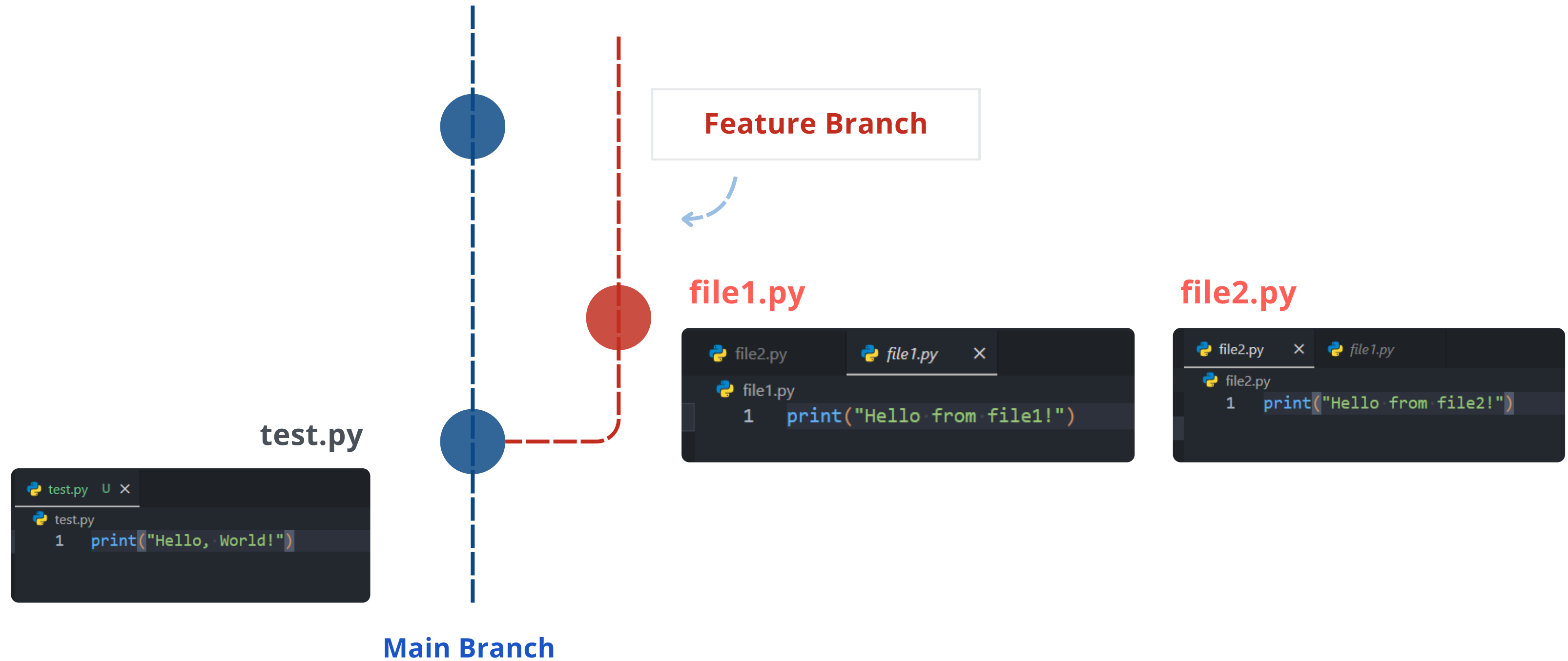
- Click the **three-dot menu** in the Source Control panel.
- Select **Push to upload the branch** and changes to GitHub.



Note:

- If the **branch** is local and not yet on the remote repository,
 - clicking "Push" will prompt
 - *"The branch [branch-name] has not been published. Would you like to publish it?"*
- Simply click "OK" to upload it to GitHub.

Visual Representation of Branching:

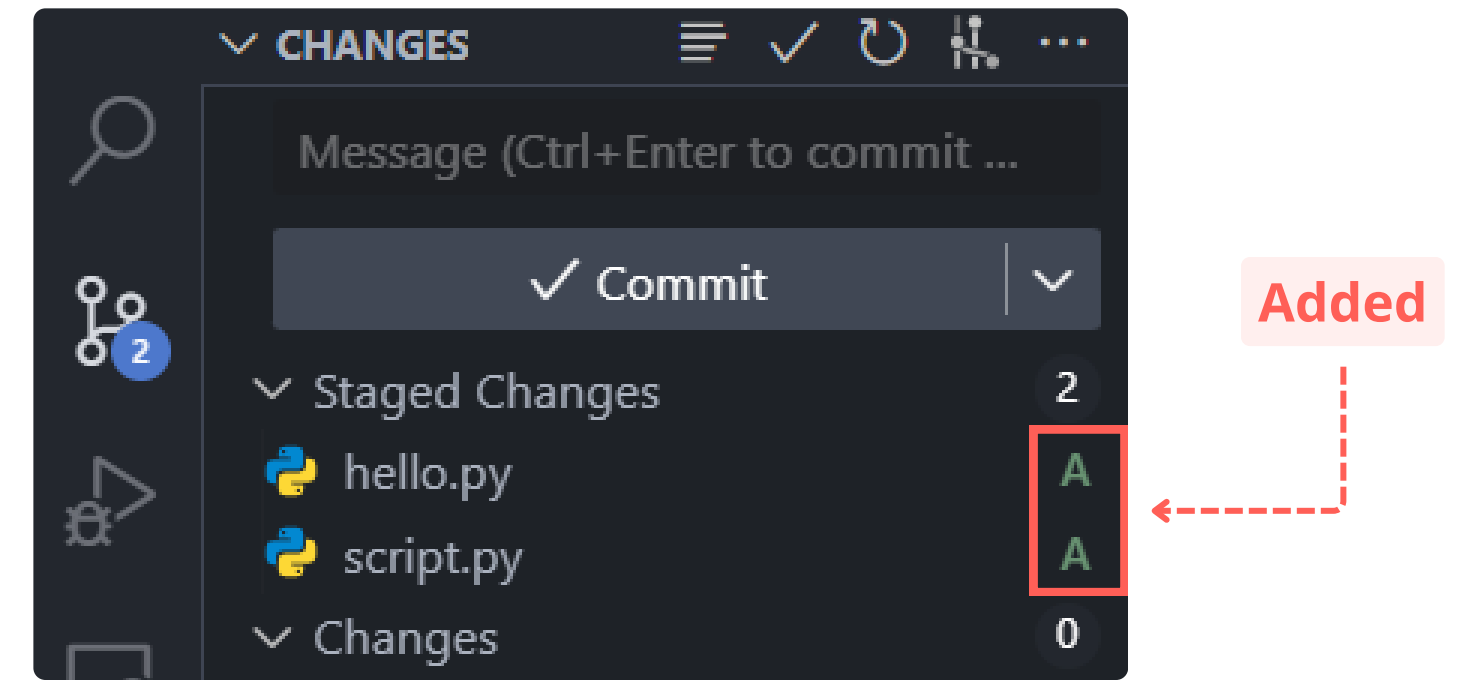


🚀 This workflow helps keep changes organized and ensures smooth collaboration!

Understanding Staged vs Unstaged Files

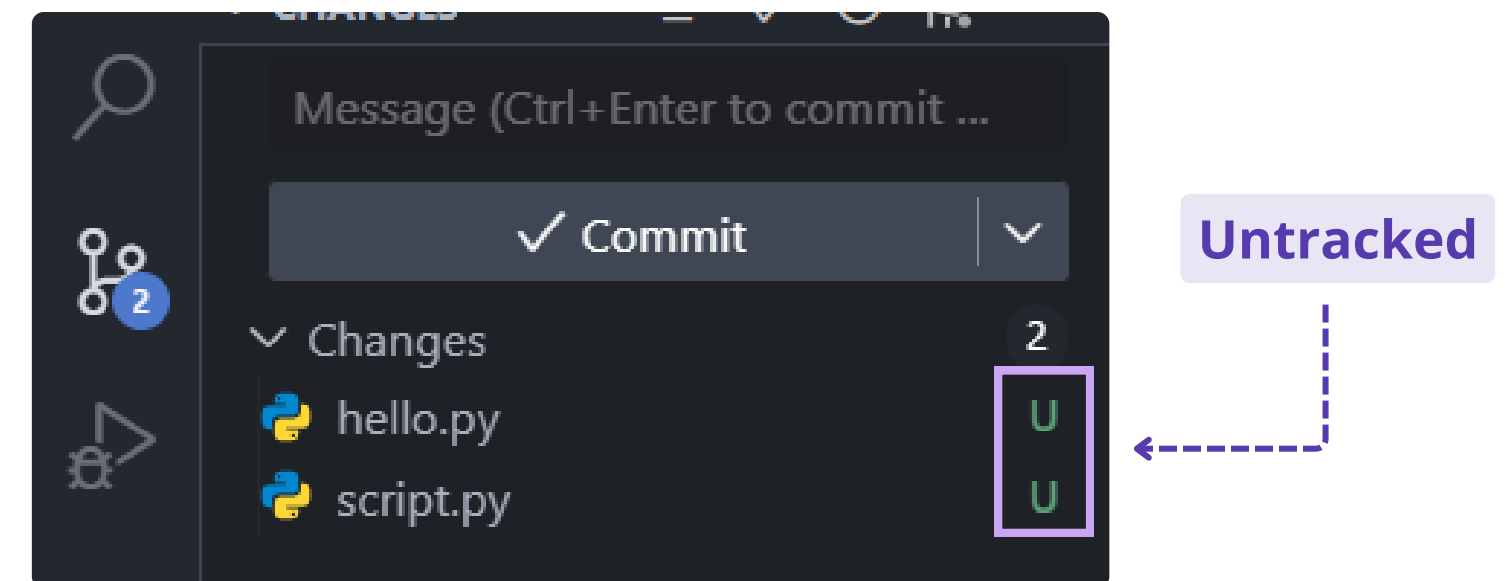
◆ Staged Files

- Staging means **selecting files for the next commit.**
- Only staged files are saved in a commit.
- In VS Code:
 - Files **marked A (Added)** are staged and ready.
 - Use the + icon to stage changes.
 - **Stage All Changes** adds everything at once.



◆ Unstaged Files

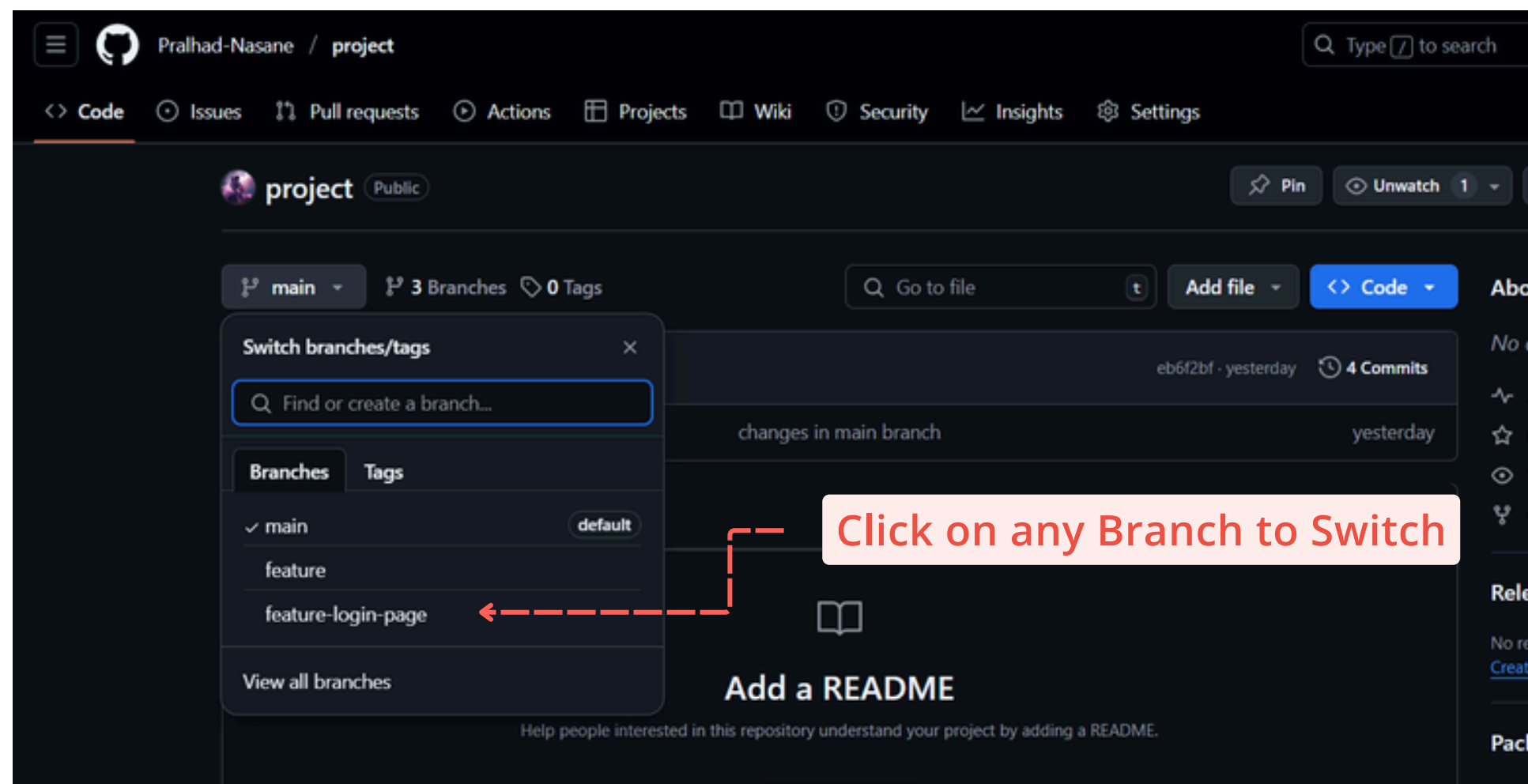
- Unstaged files are **new or modified,**
 - but **not yet marked for commit.**
- They won't be included in a commit until we stage them.
- In VS Code:
 - Files under **Changes** are unstaged.
 - **U (Untracked)** = new file not yet added to version control.



Viewing Branches on Remote (GitHub)

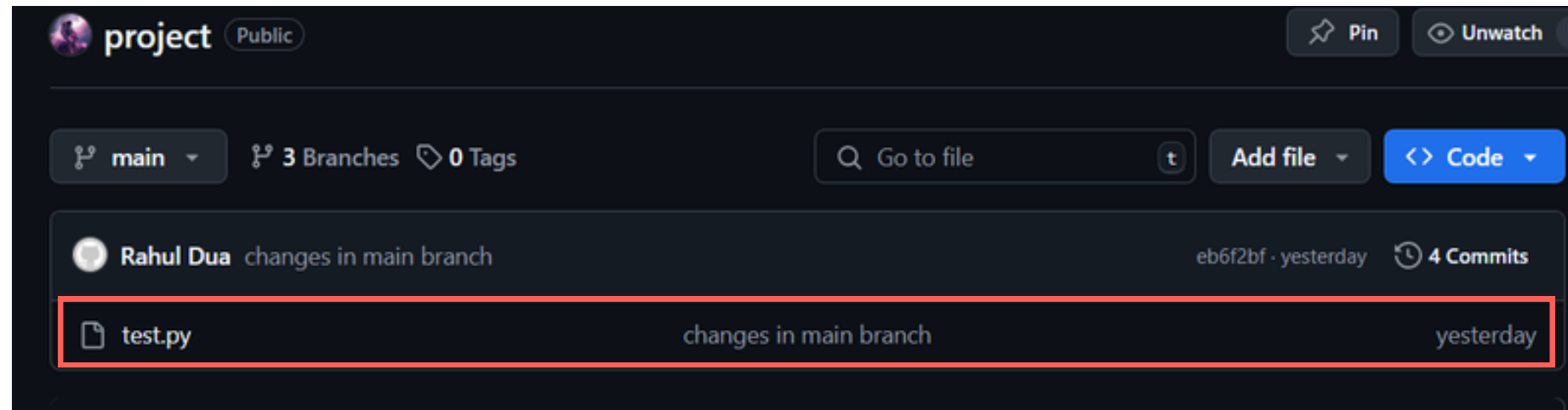
How to See Branches on GitHub

- **Go to Your Repository** → Visit github.com and open the repository you want to check.
- **Find the Branch Dropdown** → At the top-left (above the file list), you'll see the current branch name
- **Click the Branch Dropdown** → Click it to open a list of all available branches in the repository.
- **View or Switch Branches** → Click on any branch name in the list to switch to it and view its contents.



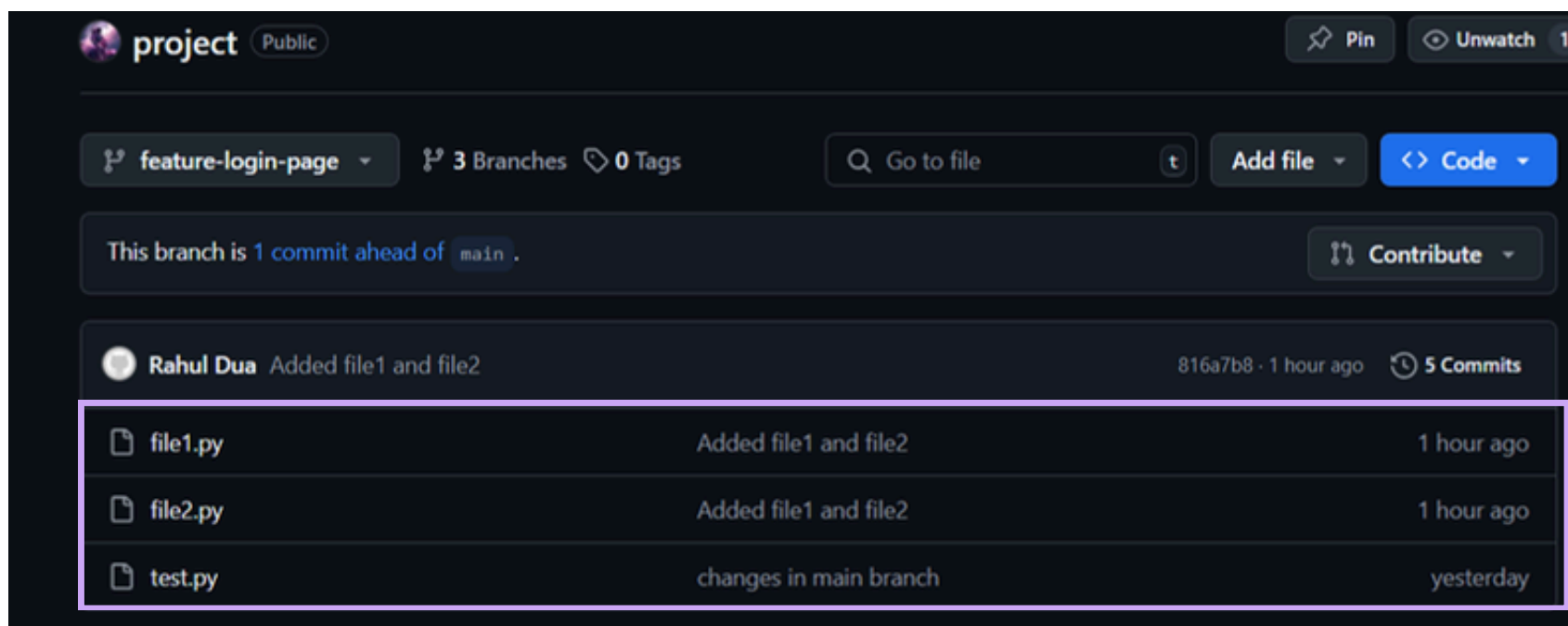
This is useful for checking different versions of your project or working in isolated features.

◆ Content of Main Branch Which has Single Python file



Contain only file1.py

◆ Content of Feature Branch with our file1.py and file2.py



Contains → file1.py, file2.py & test.py



HO: Creating and Using a Feature Branch in VS Code



Objective

- Learn how to create and switch to a new feature branch using the VS Code interface.



Instructions:

To complete this task:

- Open your cloned repository in VS Code.
- **Create a new feature branch** (e.g., **feature/add-user-auth**) using the branch menu.
- Switch to the new branch and **create/edit a file**.
- **Stage, commit, and push** the changes to GitHub.
- **Verify** that the feature branch appears in your GitHub repository.

Industry Best Practices for Branches

Avoid Direct Commits on the Main Branch

- The **main branch** should always be **stable and production-ready**.
- Never commit changes **directly to main**.
- Always create a **separate branch** for new features, bug fixes, or experiments.

Use Descriptive Branch Names

- Use **clear, meaningful names** that describe the purpose of the branch.
- Examples:
 - **feature/login-page** → New login feature
 - **bugfix/authentication** → Fixing authentication issue
 - **hotfix/payment-bug** → Urgent fix for payment issue

Why Are Branches Important?

- Branches are crucial in Git
 - they allow developers to **work independently without interfering with each other.**

Key Benefits of Branching

Parallel Development

- Multiple developers can work on **different features at the same time.**

Stable Main Branch

- **Changes stay isolated** until they are tested and reviewed.

Better Organization

- Each task or feature has its own **branch**, keeping the project **clean.**

Why is Collaborative Workflow Important?

Avoid Conflicts

Developers work on separate branches, preventing accidental overwrites.

Parallel Development

Multiple developers can work on different features at the same time, making progress faster.

Project Organization

Branches keep projects structured & manageable by separating work into features or tasks.

Code Review & Quality Assurance

Pull requests allow teams to review and approve changes before merging, ensuring better code quality.

Tracking Progress

Git tracks every change, so teams can easily monitor who did what.

Version Control

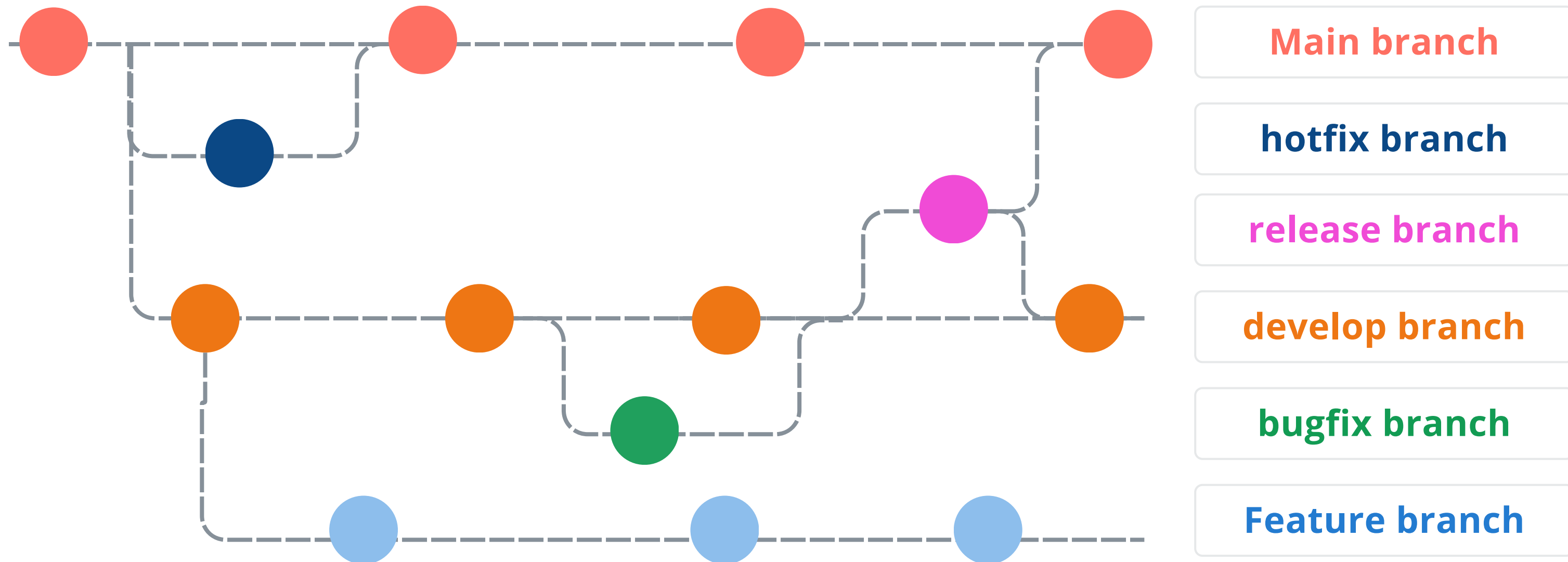
Git keeps a history of all code versions, allowing easy rollback if needed.

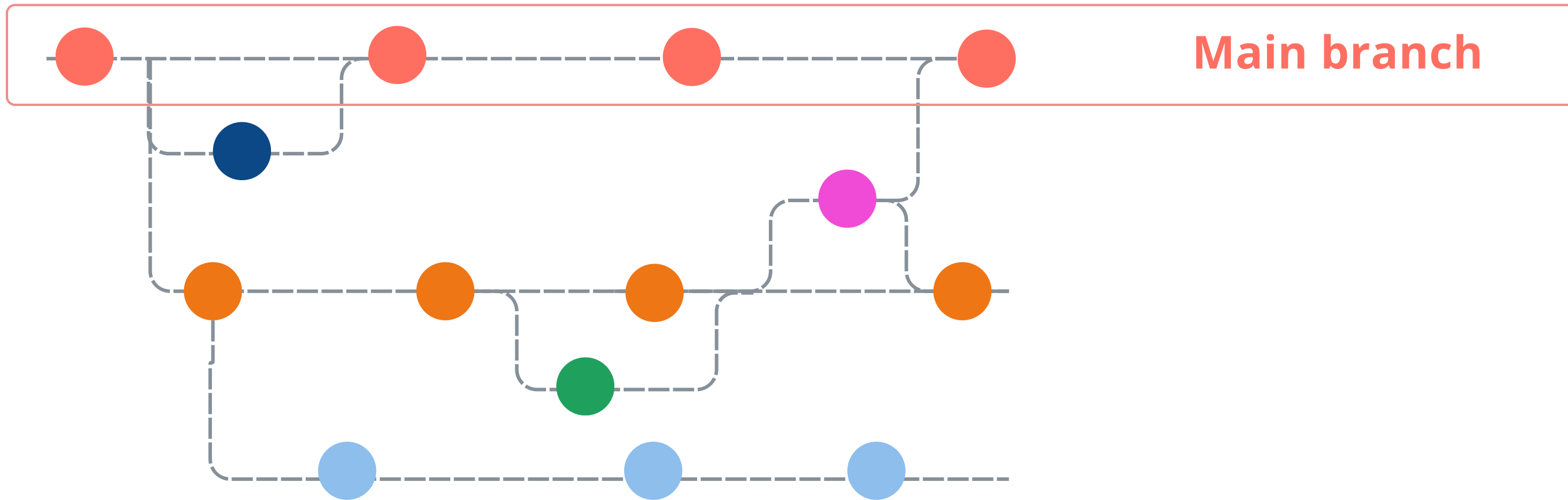
PR Workflows

- Types of Branches
- What is a Pull Request (PR)?
- Why Are Pull Requests (PRs) Required?
- Purpose of a Pull Request
- Steps in the Pull Request (PR) Workflow
- Creating Your First Pull Request (PR)
- PR Page Overview on GitHub

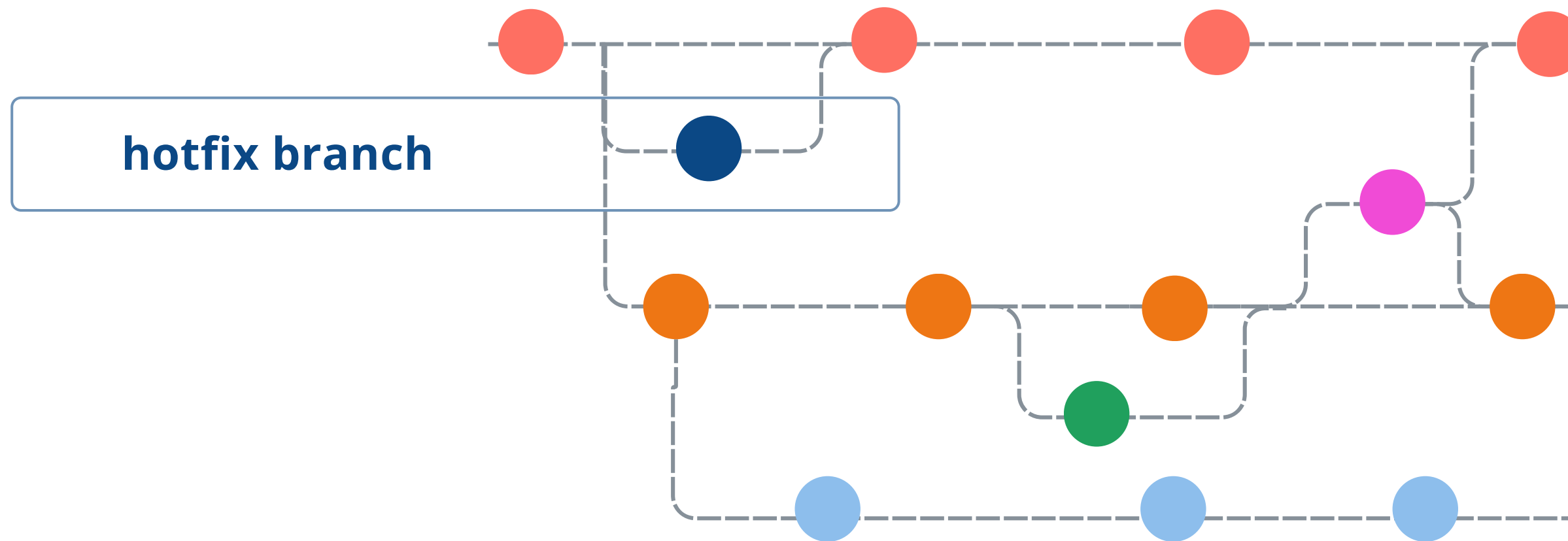
Types of Branches

- Below are various type of branches:

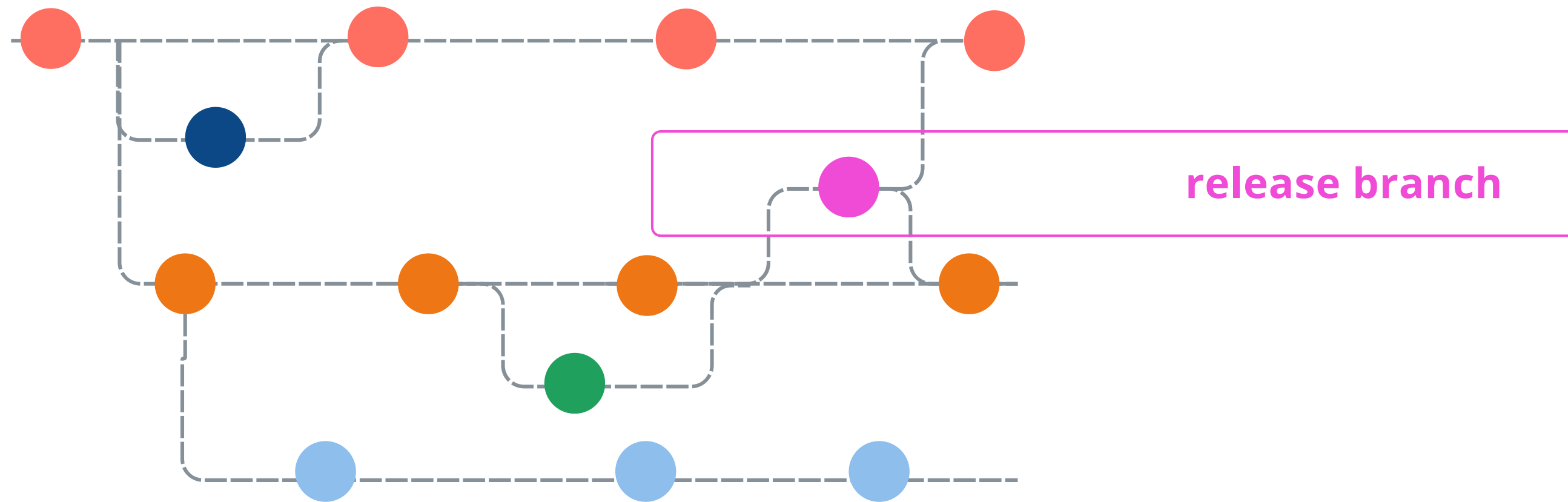




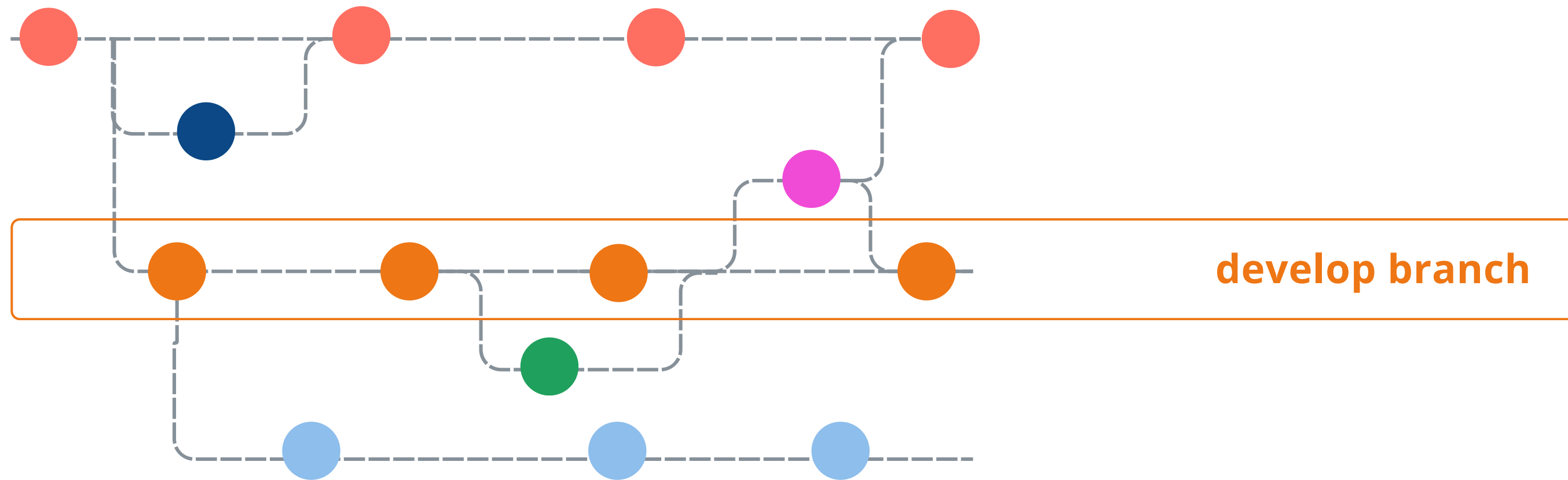
- ✓ The **primary, stable branch** where **production-ready code** lives.
- ✓ **No direct commits**—only merge tested changes.



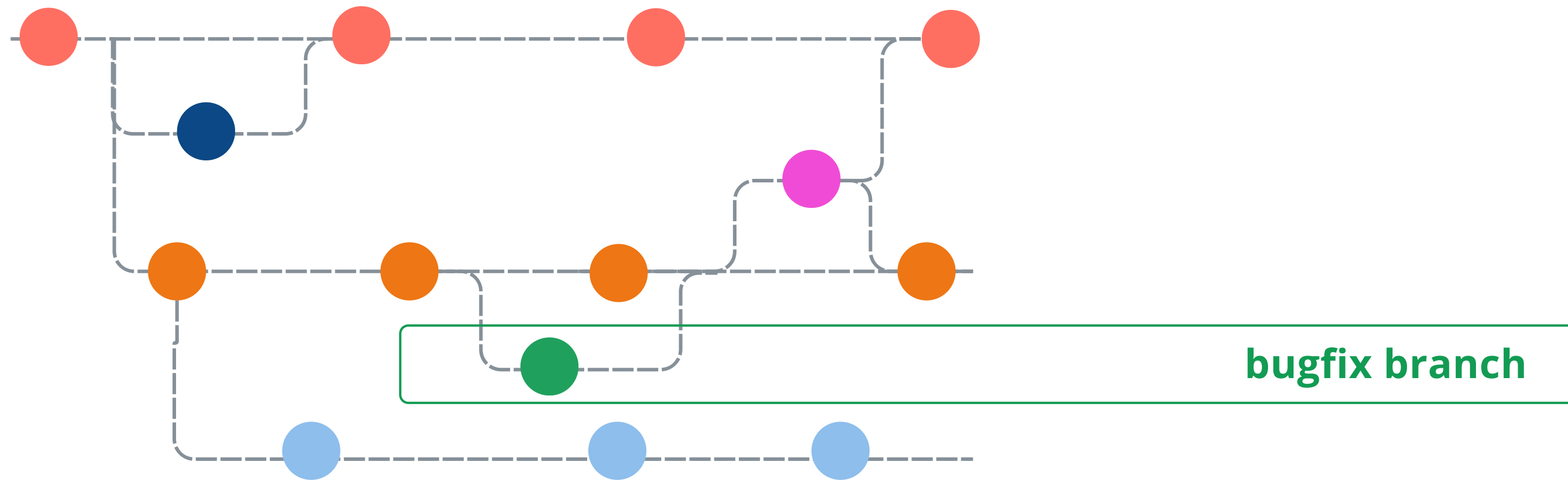
- ✓ Used for **urgent fixes** in production (e.g., **hotfix/critical-fix**).
- ✓ **Created directly from the main branch** to fix critical issues.



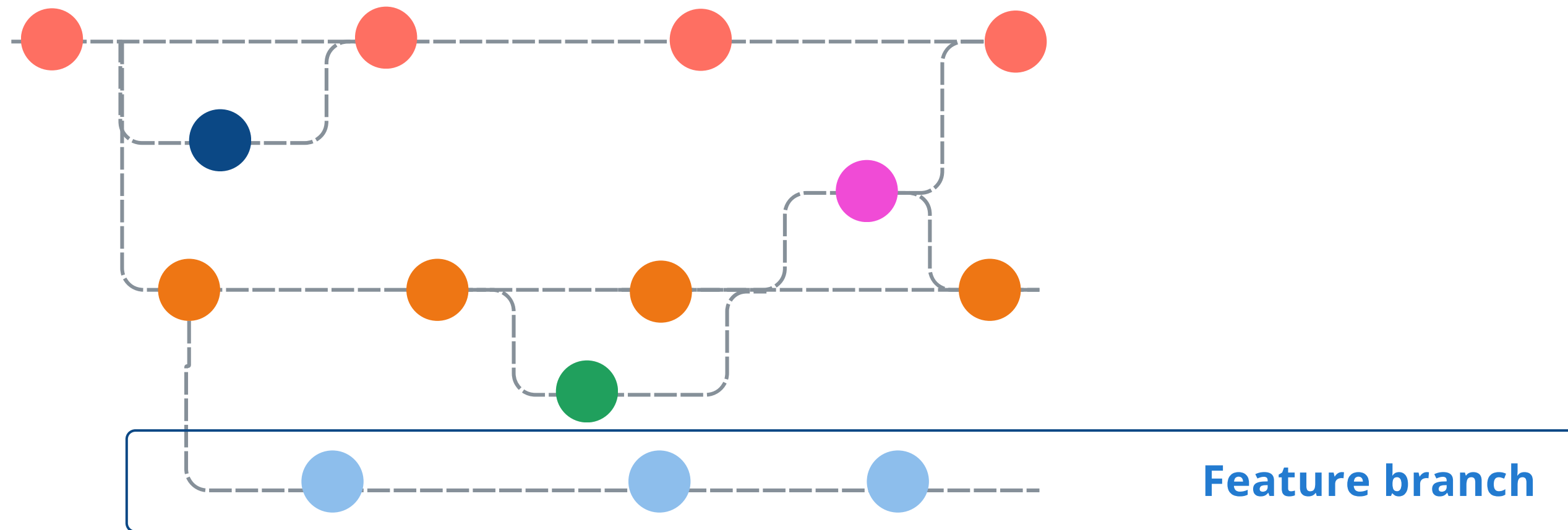
- ✓ Used for **preparing a new release**
e.g., release/v1.0.0).
- ✓ **Created** from **develop Branch**, tested, then merged into main.



- ✓ A **staging branch** where **features & bugfixes are merged** before **production**.
- ✓ **Helps** in **testing & final adjustments** before releasing.



- ✓ Used for **fixing bugs** e.g., `bugfix/fix-login-error`.
- ✓ **Created** from **main** or **feature branch**.
- ✓ Merged back **once the fix** is complete



- ✓ Used for **new features**
e.g., **feature/user-authentication**
- ✓ Created from **main** or **develop branch**.
- ✓ Merged back after development & review.

What is a Pull Request (PR)?

A Pull Request (PR) is a way to **propose code changes before adding them to the main branch** of a project.

How a Pull Request Works

1- Create a Branch

- A developer makes changes on a separate branch
- (e.g., **feature/user-authentication**)

2- Submit a Pull Request

- They open a PR to merge their branch into another branch
- (usually **main** or **develop**)

3- Team Review

- Teammates review the code, discuss changes, and suggest improvements.

4- Approval & Merge

- **After approval**
 - the changes are merged into the target branch.

PRs ensure code quality, prevent conflicts, and allow collaboration before merging changes!

Steps in the Pull Request (PR) Workflow

A **Pull Request (PR)** ensures that changes are reviewed and approved before merging into the main project.

PR Workflow Overview

- **Create a Feature Branch**
 - Developer creates a new branch (e.g., **feature/user-authentication**) from **develop**.
- **Develop the Code**
 - Write the necessary code, test it, and commit changes to the feature branch.
- **Push Changes**
 - Push the branch to the **remote repository** (GitHub, GitLab, etc.).
- **Open a Pull Request**
 - Create a **PR from** feature/user-authentication → develop **branch** for review.
- **Review & Discuss**
 - Team members **review, comment, and suggest improvements** before approval.

- **Approval & Merge to Develop**
 - Once approved, the **PR** is merged into the **develop** branch.
- **Testing in Develop Branch**
 - The feature is tested in **develop**, and if everything works, a **release branch** is created.
- **Merge to Main for Release**
 - When ready for production, a final **PR** merges **develop** into **main**.

Why Follow This Workflow?

- ✓ Keeps the main branch **stable & production-ready**.
- ✓ Allows **collaboration & feedback** before merging.
- ✓ Ensures **code quality, testing, and structured development**.

Creating Your First Pull Request (PR)

✓ Steps to Create a Pull Request (PR)

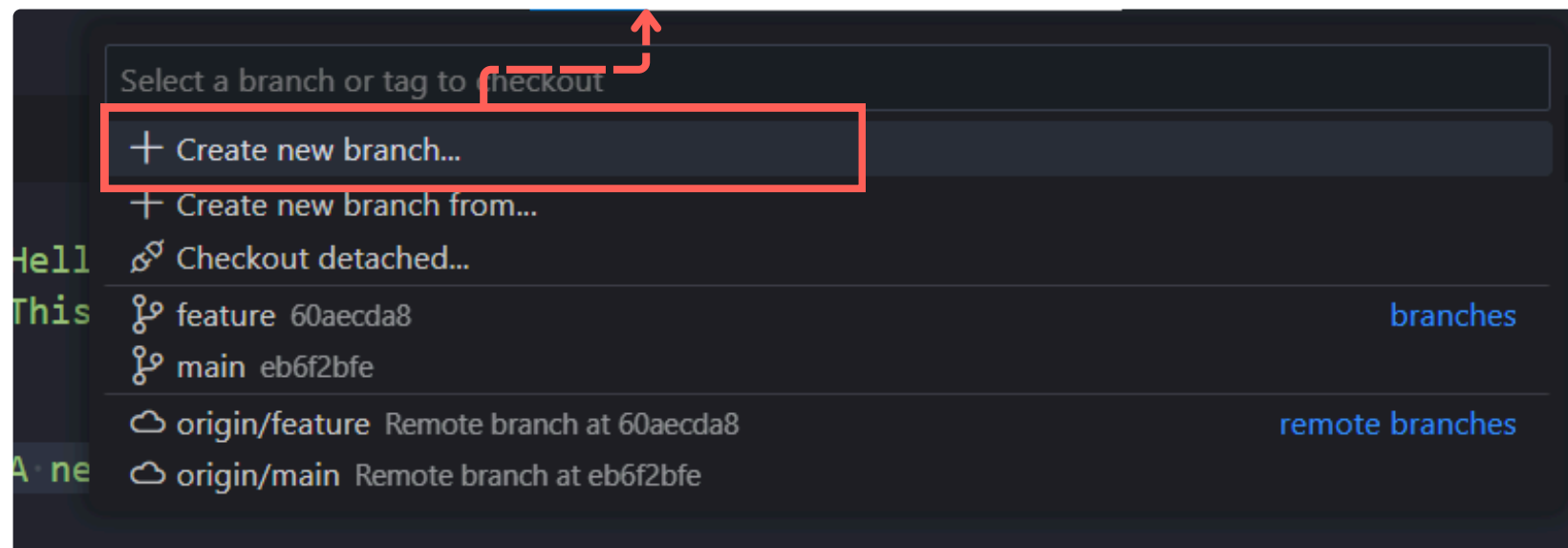
- ✓ View **Current Branch**
- ✓ Create a **Feature** Branch
- ✓ Make **Changes** (Add/Edit Files)
- ✓ **Stage** Changes
- ✓ **Commit** Changes
- ✓ **Push** Changes to Remote Repository
- ✓ Navigate to **GitHub**
- ✓ Create a New Pull Request (**PR**)
- ✓ **Review** Changes
- ✓ Add **Title & Description**
- ✓ **Submit** the Pull Request

So, before getting started the main branch contains

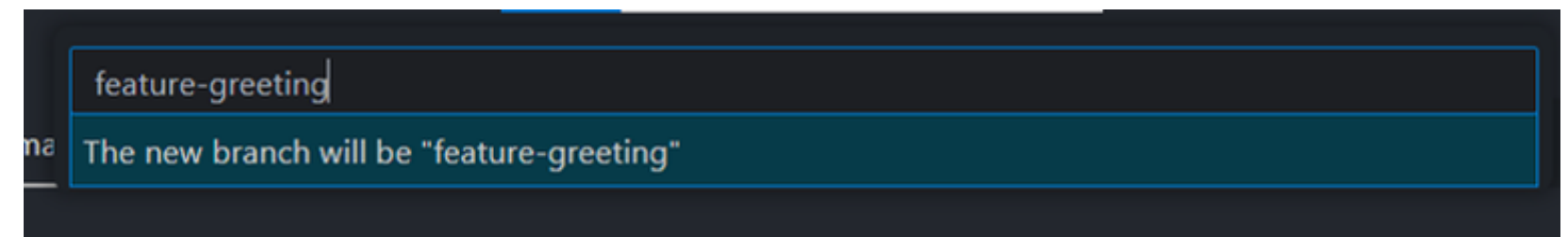
 **main.py** → `print("Welcome to the main branch.")`

1 Create a New Branch

- Click on the **current branch name** (bottom-left).
- Click on "**Create New Branch**".



- name it: **feature-greeting**

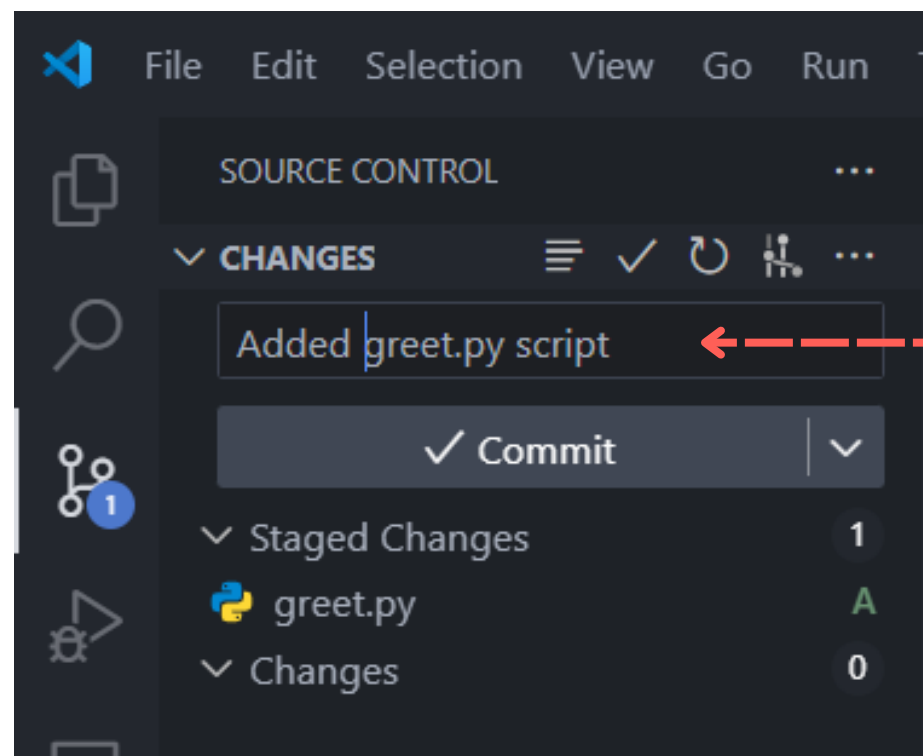


2. Add a New File in the Feature Branch

- **Right-click** inside the Explorer panel → **New File**.
- Name it **greet.py**.

```
print("Hello, Welcome to the Git Workflow.")
```

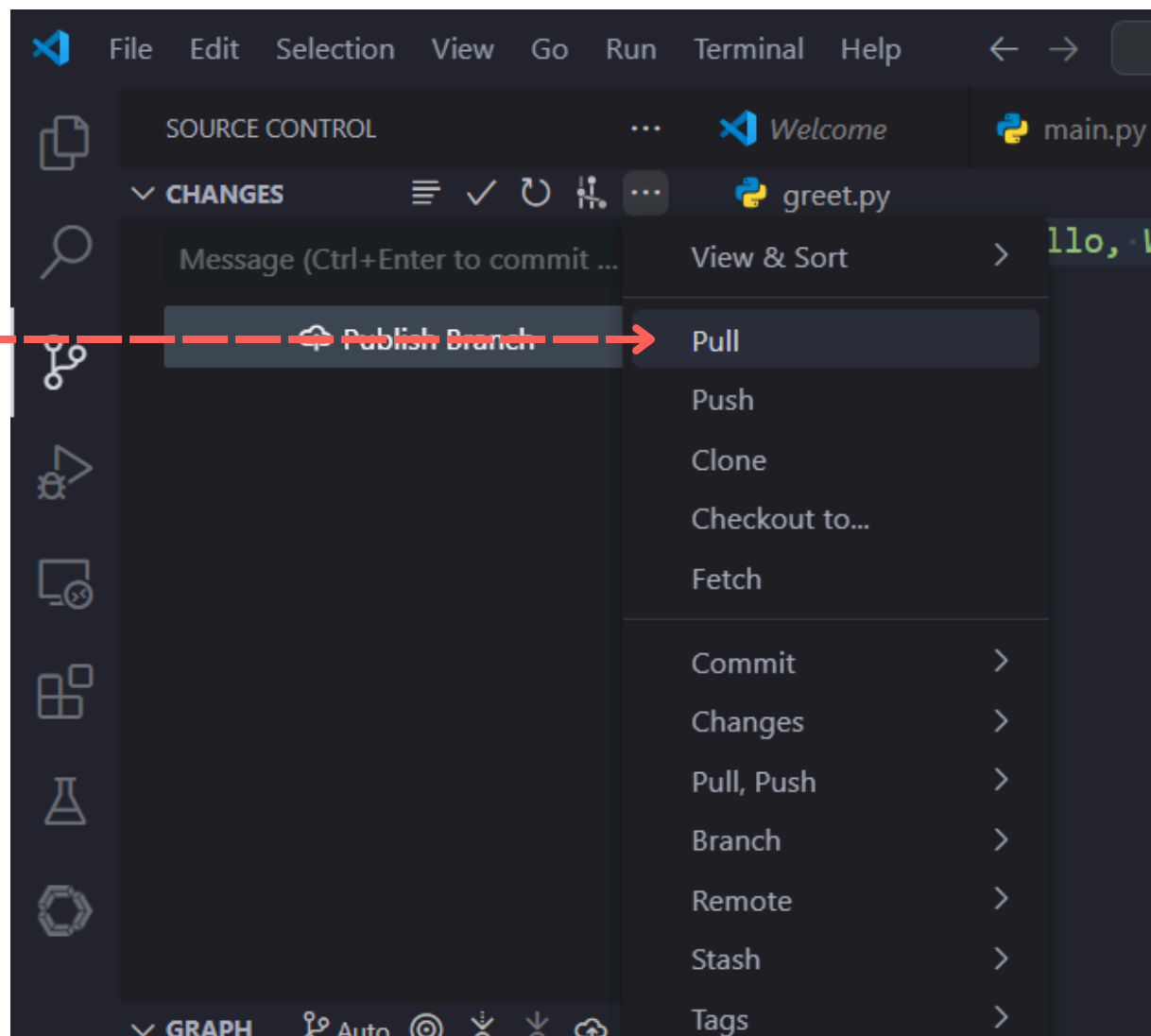
3. Stage & Commit the Changes



- **Stage file** using Source Control Panel (**Ctrl + Shift + G**)
- With **Commit** message: "**Added greet.py script** "

4. Push Changes to Remote Repository

- Click the **three-dot menu (...)**.
- Select "**Push**".
- If **prompted**, confirm creating a remote branch (feature-greeting).



5. Create a PR on GitHub

- Open **GitHub** → Go to **Pull Requests** tab → Click **New PR**
- Set **Base Branch: main** | **Compare Branch: feature-greeting**
- Add Title & Description

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main ← compare: feature-greeting ✓ Able to merge. These branches can be automatically merged.

Add a title

Add greet.py script with a greeting ← Add Title

Add a description

Write Preview

This pull request adds a new script, greet.py, to the repository.
The script introduces a greet function that prints a customized welcome message. ← Add Description

- The current main branch remains unchanged.
- Changes include the addition of greet.py in the feature branch.

Create pull request

Click to Create PR

✓ Congratulations! You've successfully created your first Pull Request! 🎉

PR Page Overview on GitHub

GamutX

The screenshot shows a GitHub Pull Request (PR) page for the repository 'Add greet.py script with a greeting #1'. The PR is created by 'Pralhad-Nasane' and aims to merge 1 commit from the 'feature-greeting' branch into the 'main' branch. The page includes a 'Conversation' tab (0), 'Commits' (1), 'Checks' (0), and 'Files changed' (1). A comment by 'Pralhad-Nasane' describes the PR's purpose and lists changes. A 'No conflicts with base branch' message indicates that merging can be performed automatically. The right sidebar shows 'Reviewers' (No reviews), 'Assignees' (No one), 'Labels' (None yet), 'Projects' (None yet), 'Milestone' (No milestone), and 'Development' (Successfully merging this pull request may close these issues).

Displays a summary of the PR

Helps ensure the code meets quality standards before merging.

Displays all modified files in the PR.

Lists all commits made in this PR.

Displays the assigned reviewers and their role in the PR process



HO: Creating and Viewing a Pull Request (PR)



Objective

- Understand how to create a Pull Request (PR) on GitHub and interpret the change comparison between branches to review and prepare code for merging.



Instructions:

To complete this task:

- Open your GitHub repository in a browser.
- **Create a pull request** to merge your feature branch into the main branch.
- **Add a title and description** to your PR.
- **View the pull request details**, including the commits and file changes.
- Review how many commits your feature branch is ahead of the base branch.



H0: Renaming a Pull Request (PR)



Objective

- Learn how to rename a pull request for better clarity and organization.



Instructions:

To complete this task:

- Go to your GitHub repository and **open the pull request** you want to rename.
- **Edit the title** to something more clear and descriptive.
- **Save the changes** and **verify** that the new title appears both on the PR page and in the pull request list

Why Are Pull Requests (PRs) Required?

A **Pull Request (PR)** ensures that changes made by multiple developers can be merged into the **main branch** in a structured way.

What PRs Help

Aspect	✓ With PRs	✗ Without PRs
• Code Review	→ Changes are reviewed by teammates or yourself	→ No review – changes go directly into main
• Bug Detection	→ Issues are often caught early during review	→ Bugs may slip through unnoticed
• Merge Conflict Handling	→ Conflicts are managed before merging	→ Conflicts often appear late and cause issues
• Team Communication	→ Comments and feedback are part of the process	→ Little to no discussion before merging
• Code Quality	→ Suggestions help improve consistency and style	→ Depends entirely on individual coding habits
• Main Branch Stability	→ Remains clean and production-ready	→ Risk of broken or untested code being merged

Purpose of a Pull Request

- **Code Review**
 - Allows team members to review and discuss changes before merging.
- **Collaboration**
 - Enables collaboration through comments, suggestions, and code reviews.
- **Version Control Best Practices**
 - Ensures only approved changes are merged, keeping the main branch stable.
- **Collaborative Development**
 - Allows multiple developers to work on different features without interfering.
- **Structured Integration:**
 - Ensures changes are merged systematically, reducing errors and conflicts.
- **Ensuring Code Quality**
 - Helps identify bugs, improve code quality, and maintain best practices.

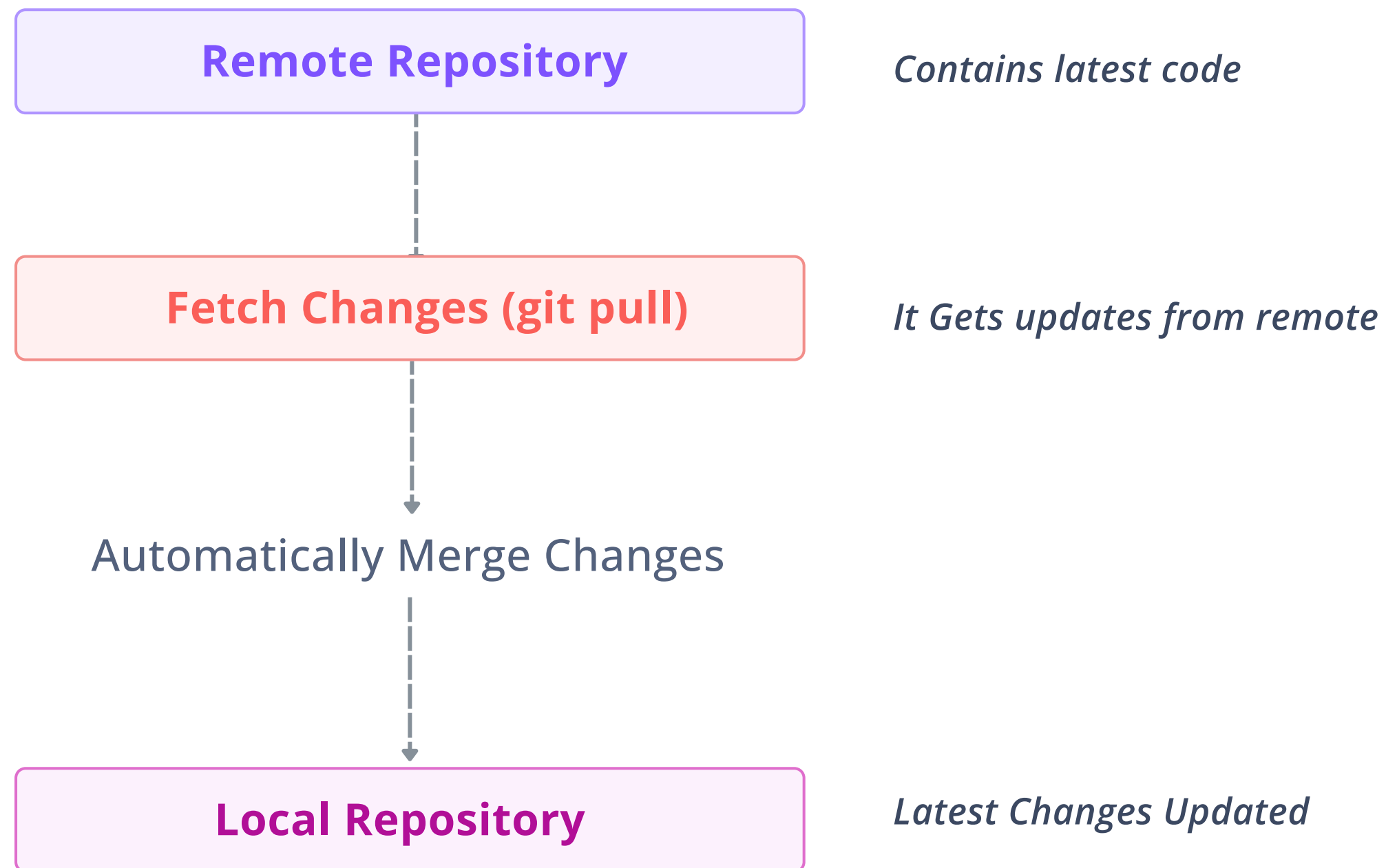
- **Conflict Resolution**
 - PRs allow early identification and resolution of merge conflicts.
 - **Documenting Change**
 - Serves as a record of why and how changes were made.
 - **Maintaining Stability**
 - Prevents bugs from being introduced to the main branch through thorough review and testing.
 - **Encouraging Collaboration**
 - Promotes teamwork and feedback during the review process.
 - **Structured Integration:**
 - Ensures changes are merged systematically, reducing errors and conflicts.
 - **Promoting Best Practices:**
 - Helps maintain a structured workflow, ensuring scalability and manageability.
- 

Git Pull

- What is Git Pull?
- Why is Git Pull Required?
- Understanding Git Pull & Push
- How to Pull in VS Code

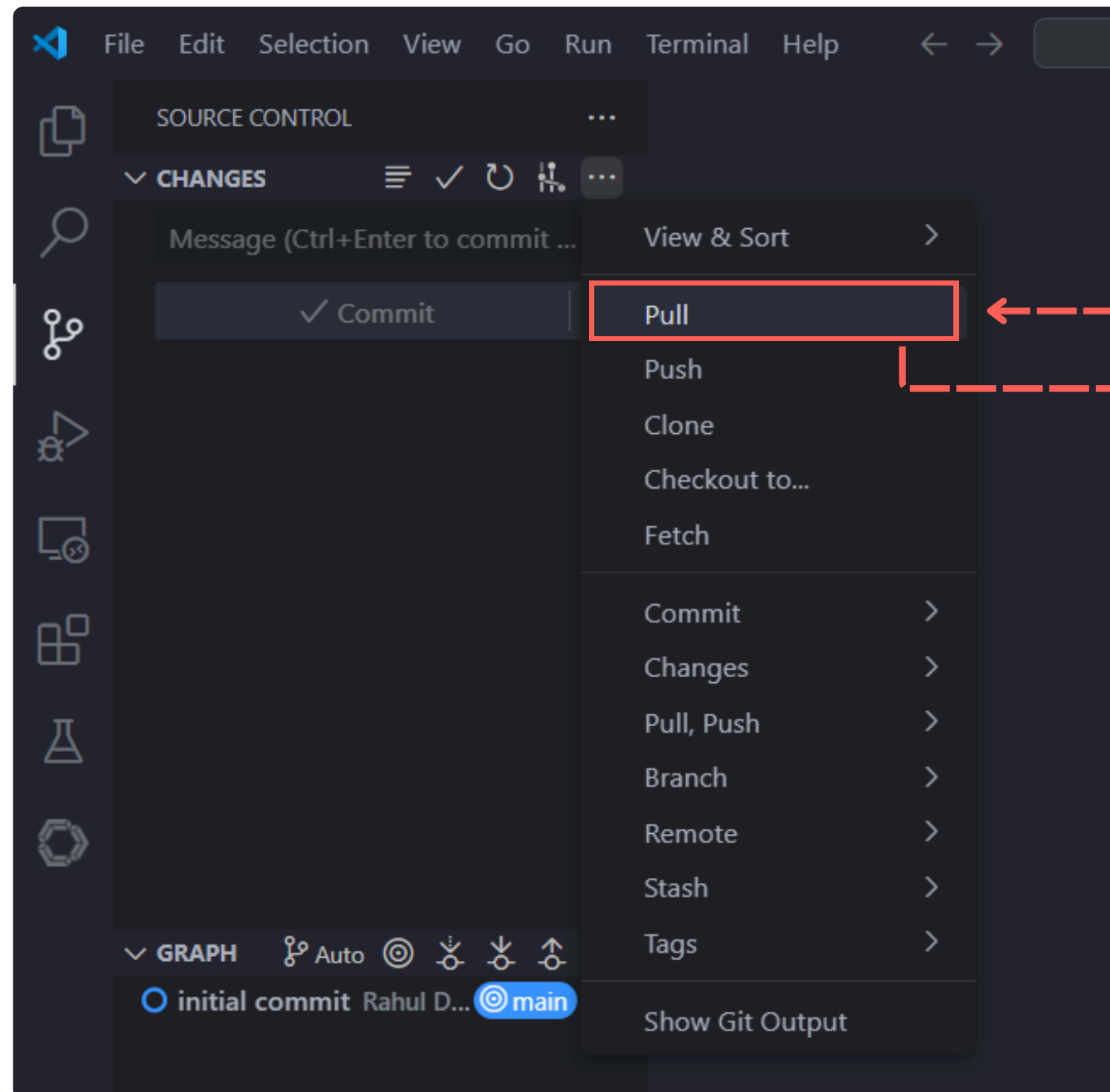
What is Git Pull?

git pull is a **command** used to **fetch and merge the latest changes** from a **remote** repository into your **local** repository.



How to Pull in VS Code

Let's See the Steps to Pull Changes Using VS Code



Steps:

- Open **VS Code** and go to the **Source Control (Git)** panel.
- At the top of the Source Control panel, **click the three-dot menu (...)**.
- From the **dropdown menu**, **click "Pull"**.
- After **pulling**, **VS Code automatically updates your files**.

This fetches and merges the latest changes

- ✓ **Edited files** – Updated code from teammates.
- ✓ **Deleted files** – Removed unnecessary files.
- ✓ **New files** – Recently added files by others.



H0: Pulling Changes from Remote to Local Repo Using VS Code



Objective

- Learn how to pull the latest changes from the remote repository to your local repository using VS Code.



Instructions:

To complete this task:

- Open **your local repository** in VS Code.
- Use the **Source Control tab** or branch menu to pull changes.
- **Authenticate** if prompted.
- **Verify** that any updates made remotely are now reflected in your local files.

Understanding Git Pull & Push

- **Git Pull** and **Git Push** are fundamental Git commands that help **sync**
 - **local repository with the remote repository.**



Command	Purpose	When to Use?
git pull	Get the latest changes from remote and merge them into local	Before making changes, to stay updated
git push	Send local commits to the remote repository	After committing, to share updates

✓ **Best Practice: Always git pull before git push to avoid conflicts!**

Why Is git pull Important?

Before you start making changes to your code, it's a good habit to run:

```
git pull
```



fetches the latest changes from the remote repository and merges them into local

Key Reasons to Use git pull

- **Prevents Conflicts**
 - Brings in any updates made by others, so you're not editing outdated code.
- **Keeps Your Code Updated**
 - Ensures your local copy reflects the latest work from the team.
- **Supports Team Collaboration**
 - Helps everyone stay on the same page and avoids overlapping changes.

📌 Good Practice

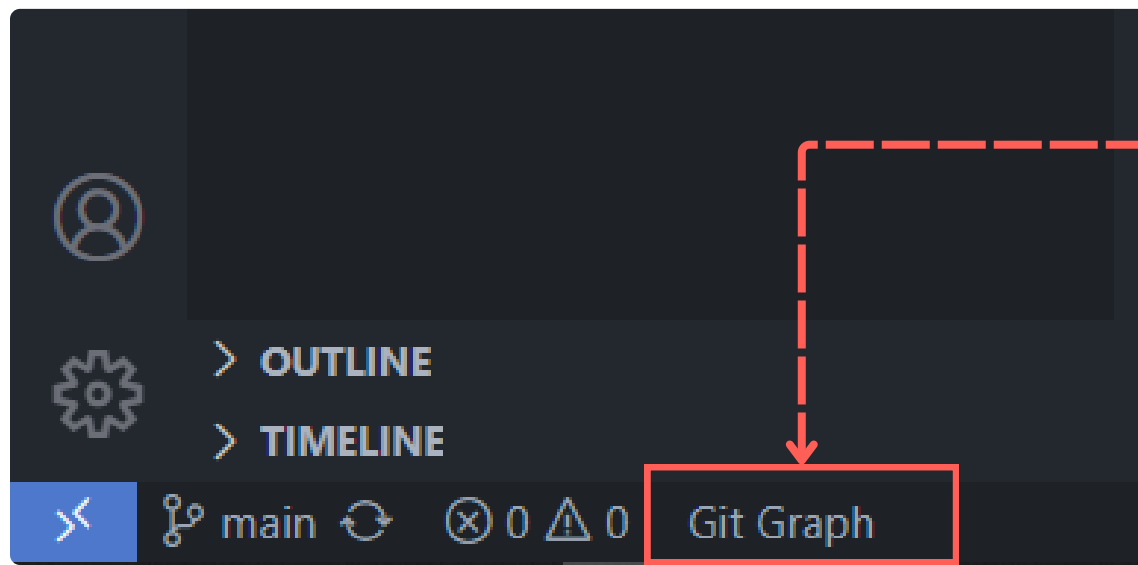
- Always run **git pull** before you start working.
- It helps prevent avoidable errors and keeps your work in sync with the team.

Git Graph in VS Code

- How to Open Git Graph
- What is Git Graph Used For?

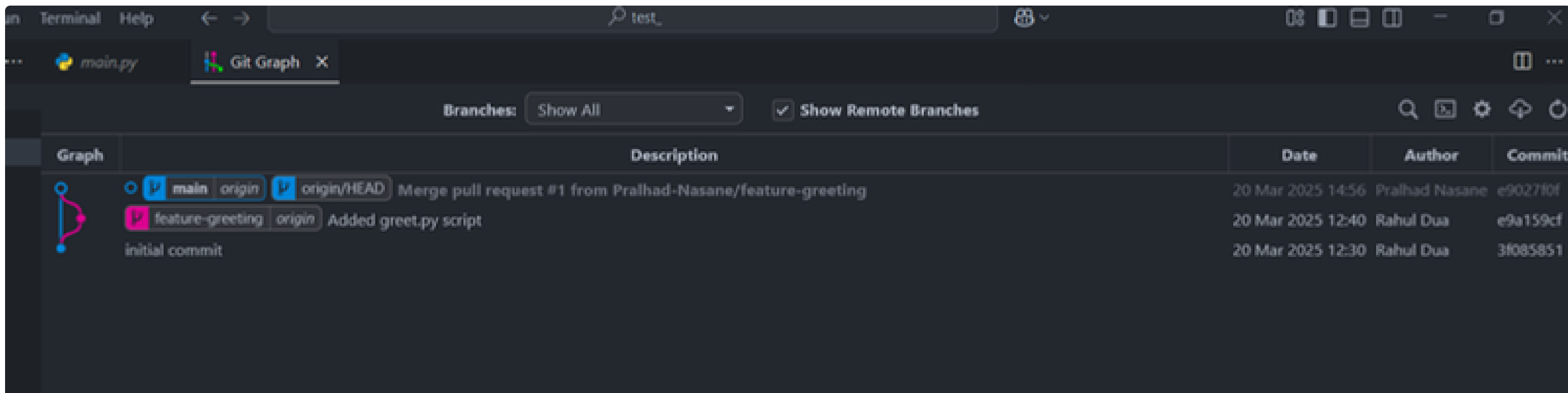
How to Open Git Graph

Open Git Graph from Status Bar



👉 Look at the **bottom-left corner** of the **VS Code window** (Status Bar).

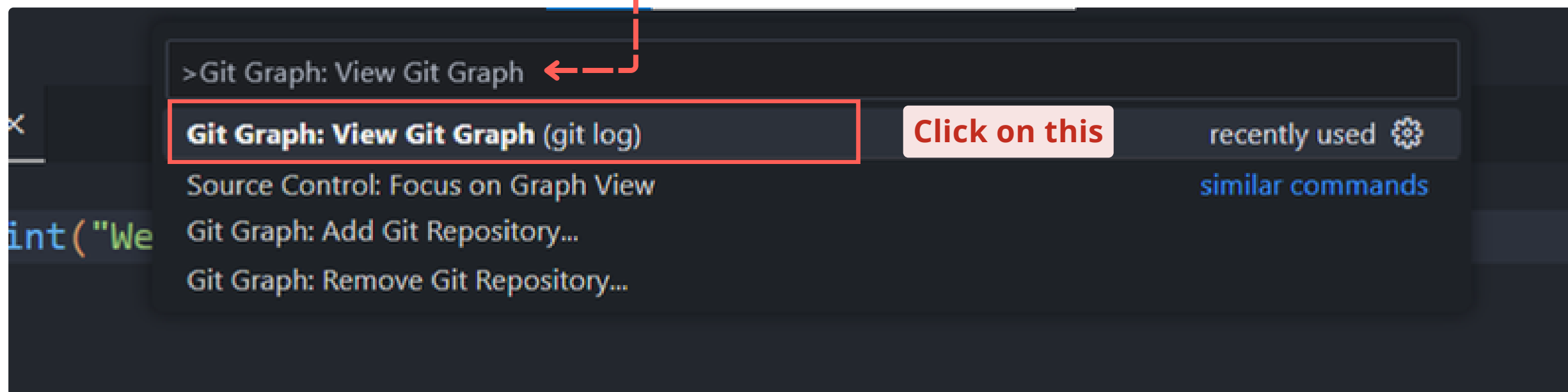
👉 **Click** on the **Git Graph** Text to open it.



Use Command Palette (Alternative Method)

👉 Press **Ctrl + Shift + P**, type:

"Git Graph: View Git Graph" → Select it.

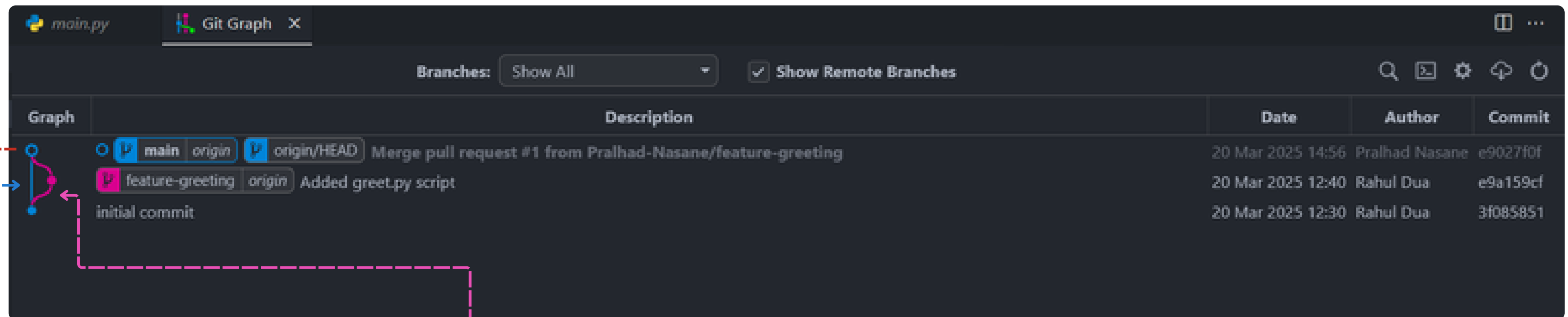


What is Git Graph Used For?

Exploring the Graph

- Branch Structure –
 - The main branch (main or master) appears as a straight line, while feature branches branch off from it.

Each dot (node) represents a commit.



Blue Line indicates Main Branch

indicates Feature Branches

View Commit Details

The screenshot shows the Git Graph interface in a dark theme. At the top, there's a tab for 'main.py' and a 'Git Graph' tab. Below the tabs, there's a 'Branches:' dropdown set to 'Show All' and a checked 'Show Remote Branches' checkbox. The main area displays a commit graph with three commits. The selected commit is highlighted with a green border. The commit details panel on the right shows the commit message, author, date, and a list of changed files.

Graph	Description	Date	Author	Commit
main origin origin/HEAD	Merge pull request #1 from Pralhad-Nasane/feature-greeting	20 Mar 2025 14:56	Pralhad Nasane	e9027f0f
feature-greeting origin	Added greet.py script	20 Mar 2025 12:40	Rahul Dua	e9a159cf
initial commit		20 Mar 2025 12:30	Rahul Dua	3f085851

Commit details for e9a159cf01ee64f046e46438f6ee8cd6ba65b6a:

- Commit: e9a159cf01ee64f046e46438f6ee8cd6ba65b6a
- Parents: 3f08585111657e974dfed21f7abc95b2c8ee29ba
- Author: Rahul Dua <rahulduaa@gmail.com>
- Committer: Rahul Dua <rahulduaa@gmail.com>
- Date: Thu Mar 20 2025 12:40:07 GMT+0530 (India Standard Time)

Added greet.py script

Changed files: greet.py

Clicking on a commit shows:

- ✓ Commit message
- ✓ Author & timestamp
- ✓ Changed files
- ✓ Diff view – See added, modified, or deleted content.

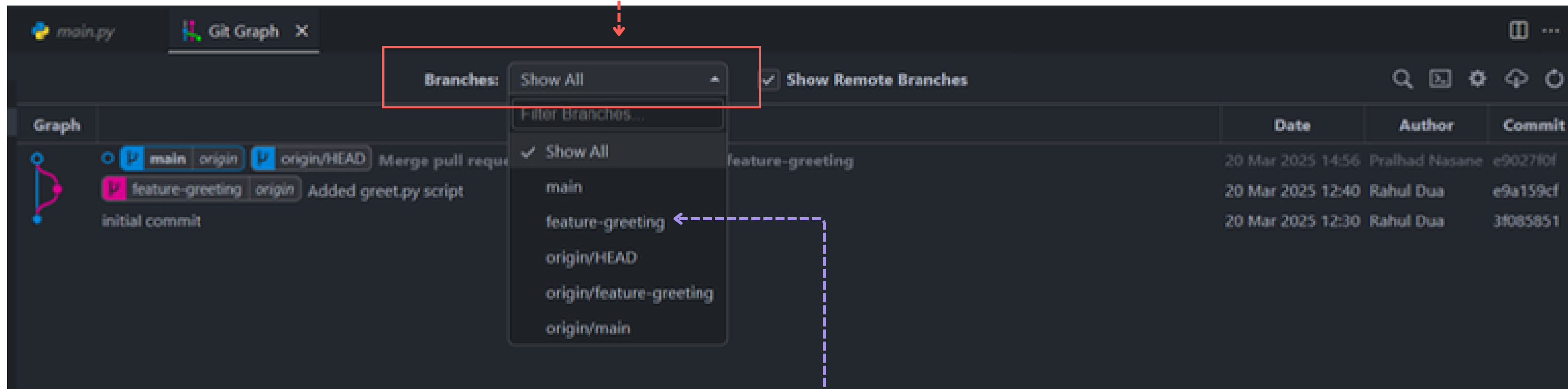
Filter Branches

- Allows you to **view commits from specific branches** instead of all branches at once.
- Helps focus on **changes in a single branch, making history easier to understand.**

How to Open:

Click on the Branches Dropdown

Located at the top of the Git Graph panel.

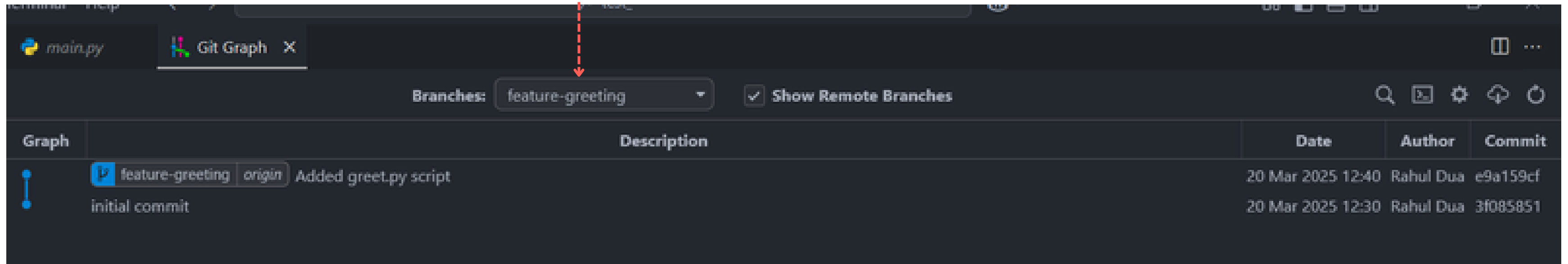


Choose a Specific Branch

Select main, **feature-greeting**, origin/main, or any other branched

Feature branch View

feature branch



Why Use Git Graph?

- Easily **track branch history**.
- Understand **how features and fixes were merged**.
- Quickly identify **who made what changes**.
- Useful for **troubleshooting merge conflicts**.

Thank You

